

Lecture 1 — IoT Introduction

Software Technology for Internet of Things | SDU MSc Software Engineering | Exam notes

1. IoT Definition

Course definition: A cyber-physical system consisting of a large number of individually and globally addressable devices which have a significant portion of their logic running elsewhere.

Common (weaker) definitions mention: networked gadgets, instrumented everyday items, or controllable-from-a-single-app devices.

Key IoT consequences:

- Siloed access — vendors want to own the entire stack
- Lots of external dependencies — integration complexity
- Lack of security — 'first to market' culture (out of scope for this course)

2. DIKW — Data to Wisdom

Level	Transform	Example (traffic light)
Data	raw	217 x v2 x 221.192.199.49 x red
Information	+ meaning	North-facing light on Niels Bohrs Alle turned red
Knowledge	+ context	The traffic light I am driving towards has turned red
Wisdom	+ experience	I better apply the brakes!

3. Logic Placement — Cloud, Fog, Edge, Mist

Tier	Location	Processing power	Latency	Privacy
Cloud	Central data centre	Highest	Highest	Lowest
Fog	Regional infrastructure	High	Medium	Medium
Edge	Local gateway/hub	Medium	Low	Higher
Mist	On the device itself	Lowest	Lowest	Highest

Key tradeoff: moving logic toward the mist reduces latency and improves privacy, but imposes severe resource constraints (memory, CPU, energy). Moving toward the cloud gives ample resources but increases latency and privacy risk.

4. Big Data

Course definition: A class of processing regimes whose primary property is that it relies on a body of data which expands at a rate where processing has become nontrivial.

Traditional definition uses the 3 V's: Volume, Velocity, Variety. Course definition focuses on the processing challenge.

Consequences of big data collection:

- More sampling hardware
- Higher network capacity use
- Larger databases and more processing
- Impacts: system design, energy consumption, bill of materials, price

5. Intended Learning Outcomes (ILOs)

Knowledge

- Describe fundamental IoT terminology
- List and describe dimensions of the wireless performance space

Skills

- Describe IoT system architecture
- Design and implement IoT systems
- Measure and present performance characteristics

Competences

- Analyse data collection requirements and map to sampling + budget management strategy
- Compare and argue for choice of IoT radio networking technologies
- Reflect on privacy and resource implications of logic placement

6. Programming Language Choices

Tier	Language(s)	Key reasons
Edge / Device	C (primary), Rust, Zig	No stop-the-world GC (real-time requirement), low memory footprint, hardware access, bit manipulation
Cloud / Fog	Elixir / Erlang (BEAM)	Massive concurrency (actor model), hot code updates, high availability, granular supervision, preemptive scheduling

Java, C#, Python, Go are excluded from edge due to mark-sweep garbage collectors causing stop-the-world pauses — incompatible with real-time device driver requirements.

7. GDPR Primer

Who: EU citizens (similar rulesets elsewhere).

When triggered: Collected data which can — alone or combined — be linked to an individual, directly or through processing.

Requirements:

- Individuals must be informed: what data, for what purpose, for how long
- Individuals must voluntarily agree (opt-in)
- Illegal to discriminate against individuals who opt out

Violations: Significant financial penalties.

8. Discussion: Intelligent vs Smart

A recurring distinction in IoT discourse:

- **Smart** — networked, remotely controllable, reacts to rules/triggers. Example: a smart bulb that turns on at sunset.
- **Intelligent** — exhibits adaptive behaviour, learns or infers from context without explicit programming. Much rarer in practice.

Most commercial 'smart' devices are not intelligent — they follow fixed rules. True intelligence requires on-device or cloud ML/AI with adaptation.

Lecture 2 — Drinking the Elixir

Software Technology for Internet of Things | SDU MSc Software Engineering | Exam notes

1. Erlang & the BEAM — Why It Exists

Developed by Ericsson in the 1980s for telephone exchange software — the only distributed systems of the era. Three driving requirements:

- Distribution — logic spread across many nodes
- Soft real-time properties — predictable, consistently low latency
- Near-zero downtime — survive software updates and hardware faults

The resulting platform has three components:

Component	Role
Erlang language	Original language; compiles to BEAM bytecode
BEAM VM	Distributed virtual machine; executes bytecode across a cluster
OTP (Open Telecom Platform)	Standard library of behaviours, supervision patterns, tooling

2. Real-World BEAM Adoption

Organisation	Use
Discord	Elixir — scaled to 5M concurrent users
WhatsApp	Erlang — each server covers 2M users
Facebook	Erlang — chat serving 100M active users
Ericsson	Erlang — GPRS/3G/4G/5G; ~40% global market share
Cisco	2M devices running Erlang (2018); 90% of internet traffic through Erlang nodes
Maersk	Measures, calculates and documents shipping CO2 emissions

Viriding's First Rule: Any sufficiently complicated concurrent program in another language contains an ad hoc, informally specified, bug-ridden, slow implementation of half of Erlang.

3. Why Elixir (over plain Erlang)

Design principle	What it means
Concurrency as 1st-class citizen	Actor model built into the language; processes are cheap and idiomatic
Documentation as 1st-class citizen	Well-documented standard library; doc generation built in
Testing as 1st-class citizen	ExUnit built in; strong culture around tests
Developer happiness	Modern syntax, good tooling (mix, hex), LiveBook notebooks

Industry domains: Network/cloud (inherits BEAM), Web (Phoenix + LiveView), Embedded (Nerves for RPi-class devices), Finance.

4. Functional Programming Concepts

Imperative

Focus: **data**. You describe *how* the result is reached — a recipe of explicit steps.

Declarative / Functional

Focus: **result**. You describe *what* the program shall achieve; control flow is implicit.

Key FP properties

- Data are **immutable** (variables can be rebound but underlying data unchanged)
- Everything is an **expression** (evaluates to a value — including if/else)
- Functions are **first-class values** (pass, return, store)
- No loops — use **recursion** instead
- **Pure functions**: output depends only on inputs; no side effects → trivially parallelisable and testable

5. BEAM Architecture

Layer	Description
Load balancer	Distributes processes across schedulers
Schedulers (one per HW thread)	Each scheduler runs on an OS thread mapped to a HW thread; pre-emptively schedules BEAM processes with fine granularity
BEAM process	Lightweight user-space process; own heap, mailbox, stack. Millions can run concurrently
Erlang Distribution Protocol	Transparent message passing between nodes (N1...N5) over the network

Elixir/Erlang compile to BEAM bytecode. Both can interoperate — call Erlang from Elixir freely.

6. Elixir Language Essentials

Basic types

Type	Notes
integer	Arbitrary size. Use div/rem for integer division. Binary/octal/hex supported.
float	64-bit IEEE floats.
atom	Global enum; name starting with colon. Equal if names equal. Not GC'd!
string	UTF-8, double quotes. Charlist uses single quotes (rarely used directly).
list	Immutable linked list. Access head/tail with hd/tl.
tuple	Immutable array, curly brackets.
map	Any-to-any key-value mapping.
pid	Process identifier.

Pattern matching

The `=` operator is a match operator, not assignment. Left side is a pattern; right side is an expression.

```
iex> {n, _, e, _} = Circle.get_bbox(circle) # destructure tuple
iex> [head | tail] = ["Bonobo", "Gorilla"] # list head/tail
iex> %{x: x_val} = %{x: 3.14, y: 2.7} # map subset match
iex> case result do
...> {:ok, msg} -> "Success: #{msg}"
...> {:error, msg} -> "Error: #{msg}"
...> end
```

Pipelines

The pipe operator `|>` passes the result of the left expression as the first argument of the right function — avoids deep nesting.

```
result =  
input("your name?")  
|> lookup(query)  
|> process()  
|> report()
```

Bang variants (e.g. `readline!`) unwrap `{:ok, value}` tuples and raise on `:error` — useful for pipeline ergonomics.

7. The Actor Model

An **actor** (called a *process* in Elixir) encapsulates: **state**, **functionality**, and a **message queue (mailbox)**.

Property	Detail
Isolation	Processes share no memory; communicate only by message passing
Concurrency	Processes run concurrently; inactive when mailbox is empty
Message order	Messages consumed FIFO; processed strictly sequentially within a process
Cheap spawning	Spawning a BEAM process is orders of magnitude cheaper than an OS thread

Synchronous (call) vs Asynchronous (cast)

Pattern	Behaviour	Use when
<code>GenServer.call</code>	Client blocks until server replies	You need a return value
<code>GenServer.cast</code>	Fire-and-forget; client continues immediately	One-way notification or state update

Minimal GenServer pattern

```
defmodule State do  
  use GenServer  
  # Client API  
  def start_link(), do: GenServer.start_link(__MODULE__, nil)  
  def get(srv), do: GenServer.call(srv, :get)  
  def set(srv, v), do: GenServer.cast(srv, {:set, v})  
  # Callbacks  
  def init(_), do: {:ok, 0}  
  def handle_call(:get, _from, state), do: {:reply, state, state}  
  def handle_cast({:set, v}, _state), do: {:noreply, v}  
end
```

8. Supervision Trees

Each process runs in a **sandbox**. A crash stays contained — it does not directly affect other processes. A **supervisor** monitors child processes and restarts them according to a strategy.

Strategy	Behaviour on child crash
<code>one_for_one</code>	Only the crashed child is restarted
<code>one_for_all</code>	All children are restarted
<code>rest_for_one</code>	The crashed child and all children listed after it are restarted

Defensive vs Offensive programming

Style	Approach	Trade-off
-------	----------	-----------

Defensive	Handle every error with if/exceptions inline	Error handling sprinkled through core logic — hard to read
Offensive (Let It Crash!)	Let the process die; supervisor restarts with fresh state	Clean core logic; relies on well-structured supervision tree

Restarting a BEAM process is cheap. If 1 in 1M processes crashes and restarts, the impact is negligible — the other 999,999 continue unaffected. Kubernetes offers similar supervision at far greater cost (container/VM granularity).

9. Nerves — Elixir on Embedded Hardware

Nerves is a framework for Raspberry Pi-class (and larger) devices. It boots a minimal Linux kernel straight into the BEAM VM — no traditional OS overhead.

Key capabilities:

- Run a BEAM cluster spanning cloud, fog, *and* edge on the same runtime
- Shift logic between layers at runtime — move computation to where the data is
- Layer interfaces are native BEAM message passing, not HTTP/REST overhead
- NervesHub provides fleet management: remote firmware updates at scale

This is why Elixir is relevant to IoT: the same language and runtime that handles cloud-scale concurrency also runs on edge hardware, with seamless inter-layer communication.

Lecture 3 — Metadata & Information Modelling

Software Technology for Internet of Things | SDU MSc Software Engineering | Exam notes

1. Motivation — The Portability Problem

Problem: Applications hardcoded to a specific building's data point UUIDs cannot be reused. Adding a second building means rewriting every application.

Desired properties:

- **Portability** — the same application binary runs on many buildings without modification.
- **Maintainability** — changing a building does not require changing any application.

Solution: The Narrow Waist

A shared **information model** sits between applications and buildings. Applications query the model; the model maps to the building's concrete data points (UUIDs). This is the same 'narrow waist' design as the TCP/IP stack.

Layer	Role
Application 1 ... n	Queries the model using a standard interface; never knows building specifics
Information model + interface	The narrow waist — shared vocabulary and query API
Building 1 ... m	Populated into the model; exposes sensors, actuators, setpoints as UUID-named streams

2. Storage Options Overview

Option	Underlying structure	Notes
Timeseries DB	B+ tree per timeseries, indexed by timestamp	Efficient range lookups; maintains min/max/average. Good for raw sensor streams.
Relational DB	Tables with foreign keys (room, floor, building, person...)	Familiar but rigid schema; hard to extend without migrations.
Object-oriented DB	Classes with typed references (Building → Floor[] → Room)	Natural OO mapping; limited industry adoption.
RDF Ontology	Triple store (subject × predicate × object)	Very expressive; machine-readable semantics; focus of Part 1.
Property Graph (Neo4j)	Nodes + typed directed edges, both with key-value properties	Graph-idiomatic queries; fast traversal; focus of Part 2.

3. RDF — Resource Description Framework

Everything is expressed as **triples**: **subject × predicate × object**. A model is a set ("store") of triples. Each triple is a **fact** — the information lives in the relationships, not in the names.

Subject	Predicate	Object
Marge	parent of	Bart
Marge	is a	Female
Bart	is a	Male
Female	subclass of	Human

Male	subclass of	Human
------	-------------	-------

Namespaces & CURIs

RDF uses URI-based namespaces to avoid name collisions. Compact URIs (CURIs) abbreviate them with a prefix, e.g.:

- **brick**: maps to <https://brickschema.org/schema/Brick#>
- **foaf**: maps to <http://xmlns.com/foaf/0.1/>
- **rdfs**: maps to <http://www.w3.org/2000/01/rdf-schema#>

Turtle (.ttl) — serialisation format

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix brick: <https://brickschema.org/schema/1.1.0/Brick#> .
@prefix demo: <http://ontologies.sdu.dk/Demo#> .

demo:floor a brick:Floor .
demo:room a brick:Room ;
rdfs:label "U181" ;
brick:isLocatedIn demo:floor .
```

Ontologies

An ontology is a *formal naming and definition of types, properties, and relationships* for a specific domain. Key point: ontologies define a **shared vocabulary** that machines can interpret unambiguously — unlike informal human conventions.

Humans adapt when vocabulary is ambiguous; machines fault. Programmers are 'machine builders' — once deployed, a system carries all the model interpretations embedded at build time.

4. OWL — Web Ontology Language

OWL extends RDF/RDFS with richer formal semantics:

OWL feature	What it enables
rdfs:subClassOf / subPropertyOf	Type hierarchy — Office is a subclass of Room is a subclass of owl:Class
owl:ObjectProperty	Relationship pointing to another node (not a literal value)
owl:DatatypeProperty	Relationship pointing to a literal (int, string, dateTime...)
rdfs:domain / rdfs:range	Restrict which types are allowed as subject/object of a relationship
owl:equivalentClass	Bridge two ontologies — brick:Room ≡ my:Room
owl:maxCardinality	Cardinality constraint — a Person has at most 1 mother
Relationship inferences	Transitive, symmetric, inverse relationships

5. SPARQL — Query Language for RDF

SPARQL operates at the **triple level**. Variables are prefixed with ?; pattern matching finds all triples satisfying the WHERE clause.

SELECT example — find all rooms with temperature sensors, setpoints, actuators


```

SELECT DISTINCT ?room_name ?sensor_uuid ?setpoint_uuid ?actuator_uuid
WHERE {
  ?room rdf:type/brick:subClassOf* brick:Room .
  ?sensor rdf:type/brick:subClassOf* brick:Temperature_Sensor .
  ?setpoint rdf:type/brick:subClassOf* brick:Temperature_Setpoint .
  ?actuator rdf:type/brick:subClassOf* brick:Radiator_Valve_Position .
  ?sensor brick:pointOf ?room .
  ?setpoint brick:pointOf ?room .
  ?actuator brick:pointOf ?room .
  ?room brick:label ?room_name .
  ?sensor brick:label ?sensor_uuid .
}

```

SPARQL query complexity

Most library implementations use **repeated full table scans** — evaluation time grows extremely fast with query complexity. Dedicated RDF DBMS add indexes (path × length, subject × predicate, etc.) but full-table scan is always the fallback.

Mitigation strategies: add indexes, drop full language support for efficiency, or switch to a property graph model (Neo4j) which offers index-free adjacency for graph traversal.

6. Reasoners & Validators

Reasoners

Apply inference rules to derive new facts from existing ones. Test whether data conforms to the rules of the ontology.

- Given: Marge is Female + Marge isParentOf Maggie → infer: Marge isMotherOf Maggie
- Given: entity is a TemperatureSensor + TemperatureSensor subClassOf Sensor → infer: entity is a Sensor

When to apply: on insert, before querying, or during querying. Pre-applying reduces query complexity.

Validators (SHACL)

- brick:isLocatedIn requires subject = Equipment, object = Location
- Every person:Human has exactly 2 parents
- birthdate must have type xsd:dateTime

Without a validator: spelling mistakes accepted, entities can connect arbitrarily, floating nodes are allowed.

7. Open vs Closed World Assumption

Assumption	Definition	Consequence
Closed World (CWA)	Any true statement is known to be true	Can fully validate — all properties can be checked. But requires modeling everything, which can be infinite/impractical.
Open World (OWA)	A statement may be true even if not known to be true	Cannot verify a property holds (unknown ≠ false). Can only check that no restrictions are violated.
Partially Open World	Declare some properties as nullable/optional to allow practical modeling	Stays formally closed but permits incomplete data (e.g. parent fields optional).

OWL uses the Open World Assumption. SHACL adds Closed-World-style validation on top of RDF. Together they give both expressiveness and the ability to validate.

8. Property Graphs & Neo4j

Concept	Definition
Node (Vertex)	Has a set of labels (types), a key-value property store, and maps to outgoing/incoming relationships
Edge (Relationship)	Has a single type, a key-value property store, a source node and a destination node

Path	A sequence of connected edges from one node to another; used for distance/reachability queries
Cycle	A path whose source and destination are the same node. DAGs (Directed Acyclic Graphs) have no cycles.
Property Graph	A graph where both nodes and edges carry key-value properties
Index-free adjacency	Edges store direct pointers to neighbouring nodes — traversal is O(1) per hop, not a table scan

Cypher (OpenCypher) query syntax

Nodes in **(parentheses)**, edges as **ASCII arrows**, variables and types in the pattern:

```
# Anatomy: (variable:Label {prop: value}) -[:EDGE_TYPE]-> (variable)

# All father-daughter pairs:
MATCH (father:Person {sex: 'male'}),
      (daughter:Person {sex: 'female'}),
      (father) -[:parentOf]-> (daughter)
RETURN father.name, daughter.name

# Same-sex parent-child pairs:
MATCH (parent:Person)-[:parentOf]->(child:Person)
WHERE parent.sex = child.sex
RETURN parent.name, child.name
```

Use **EXPLAIN** for the query execution plan and **PROFILE** for timing — same concept as relational DBs. Extensions: APOC procedures, spatial functions, Neosemantics (SHACL validation), GraphQL adapter.

9. RDF / SPARQL vs Neo4j / Cypher — Comparison

Property	RDF + SPARQL	Neo4j + Cypher
Type	Open standard (multiple implementations)	Specific product (free + enterprise editions)
Underlying model	Triple store	Property graph
Abstraction unit	Fact (triple)	Node / edge
Schema support	Strong (OWL + SHACL)	Weak (enterprise only for full constraints)
Reasoner support	Yes (select implementations)	Theoretical only
Query pattern	Restriction on triples	Match pattern on nodes/edges
Path direction	Unidirectional	Bidirectional
Evaluation speed	Slower (full table scan fallback)	Fast (index-free adjacency)
Cross-schema	Consistent across ontologies	N/A

GraphQL is NOT a graph query language despite the name — it lacks graph-idiomatic concepts (path matching, graph algorithms). Real graph query languages: Cypher (OpenCypher) and Gremlin.

Lecture 4 — To C or Not to C

Software Technology for Internet of Things | SDU MSc Software Engineering | Exam notes

1. Why C for Embedded / IoT?

No garbage collector — GC causes unpredictable stop-the-world pauses, incompatible with real-time device-driver requirements.

Additional reasons:

- **Fast with low memory footprint** — critical on resource-constrained devices
- **Direct hardware access** — bit-level manipulation, pointer arithmetic, memory-mapped I/O
- **Essential abstractions without fluff** — functions, stack, sequence/choice/repetition/indirection in both data and logic
- **Simple language** — most of it teachable in one module

Alternatives considered: Rust and Zig. Both lack GC and are suitable, but C was chosen for simplicity and ubiquity on the ESP32.

2. Processor & Memory Model

Region	Contents	Notes
MM I/O	Memory-mapped peripheral registers	Reading/writing specific addresses controls hardware peripherals
Code	Compiled instructions (read-only at runtime)	Program counter (PC) steps through this region
Heap	Dynamically allocated data (malloc/free)	Grows upward; must be manually managed
Stack	Local variables, function call frames, return addresses	Grows downward; automatically managed via push/pop
Registers	PC, general-purpose (r0, r1...), named vars (sum, i, addr...)	Fastest storage — directly used by ALU instructions

The processor executes a fetch-decode-execute cycle: load the instruction at PC, decode it, execute it, advance PC. The assembly walkthrough in the lecture demonstrates summing an array: jeq (jump-if-equal), load, add, addi, store, goto.

3. Build Phases: C vs Java

Stage	Java	C
Source compilation	.java → .class (JVM bytecode) per file	.c → .o (object file, machine code) per file
Packaging / Linking	.class files archived into a .jar	.o files linked into a single executable binary
Execution	JVM interprets / JIT-compiles bytecode at runtime	Runs directly on hardware — no runtime VM
Pre-compilation	N/A	C preprocessor runs first: #define, #include, #ifdef expansion

The C build pipeline is: preprocess → compile (.c → .o) → link (.o → binary). The -D flag passes macro definitions from the command line (e.g. -DMULTIPLIER=3).

4. Basic Abstractions — Data & Logic

Concept	Data construct	Logic construct
Repetition	array	while / do-while / for

Sequence	struct	; (statement separator) + function calls
Choice	union	if / switch
Indirection	pointer	pointer (function pointers)

Primitive types

Size guarantees: $\text{sizeof}(\text{char}) \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long})$; $\text{sizeof}(\text{float}) \leq \text{sizeof}(\text{double})$. Booleans are just integers (0 = false). Strings are zero-terminated char arrays.

5. Data Constructs

typedef

Creates an alias for a type. Convention: suffix with `_t`.

```
typedef unsigned short uint16_t;
uint16_t var = 42;
// var-- wraps to 65535 (unsigned!)
```

enum

Named integer constants. Values auto-increment unless overridden. Useful for state machines and categories.

```
typedef enum {
    CAT_UNKNOWN, // 0
    CAT_STRING, // 1
    CAT_INTEGER=10, // 10
    CAT_FLOAT, // 11
} category_t;
```

struct

Groups named fields into one compound type. Fields accessed with `.` (value) or `->` (pointer).

```
typedef struct {
    double x;
    double y;
} point_t;

point_t p = {0, 0};
p.x = 3.14;

point_t* pp = &p;
pp->y = 2.7;
```

union

All fields share the **same memory location**. Size = size of largest field. Writing one field and reading another reinterprets the bytes — useful for type-punning in embedded protocols.

```
typedef union {
    double reading;
    int count;
} payload_t;

payload_t p = {.reading=3.14};
p.count = 42; // overwrites bytes
```

Dynamic memory

malloc(n) allocates n bytes on the heap and returns a pointer. **free(ptr)** releases it. Forgetting to free causes a memory leak; freeing twice causes undefined behaviour.

```
point_t* pt =
    (point_t*) malloc(sizeof(*pt));
pt->x = 1.0;
free(pt); // always free!
```

Arrays & uninitialized memory

Stack arrays are **not zero-initialised** — they contain whatever garbage was in memory. Always initialise before use.

```
int vals[10]; // garbage!
// vs heap (also not zeroed):
int* v = malloc(10*sizeof(int));
```

6. Pointers

A pointer variable holds an **address** (an integer). Key operators:

- **&x** — address-of: gives the address where x is stored
- ***ptr** — dereference: gives the value at the address in ptr
- **ptr->field** — shorthand for `(*ptr).field` on struct/union pointers
- **ptr + n** — advances by $n \times \text{sizeof}(*\text{ptr})$ bytes (pointer arithmetic)
- **(void*) cast** — converts between pointer types; does not change address

Segfault: dereferencing a pointer outside your valid allocations (NULL, freed memory, out-of-bounds) triggers a segmentation fault.

Function pointers

Functions can be stored in pointer variables and passed as arguments — the C equivalent of higher-order functions.

```
typedef int (*int2int_fn)(int);

int incr(int i) { return i+1; }

void map(int data[], int len, int2int_fn fn) {
    for (int i=0; i<len; i++) data[i] = fn(data[i]);
}

map(data, 11, &incr); // pass function by address
```

7. Logic Constructs

Branching

```
if (i%2==0) {
    printf("even\n");
} else {
    printf("odd\n");
}
// ternary shorthand:
printf(i%2==0 ? "even" : "odd");
```

Switch

Matches an integer expression; **break** exits the case. Missing break causes fall-through. **default** handles unmatched values.

```
switch (dir) {
    case NORTH: ...; break;
    case SOUTH: ...; break;
    default: ...;
}
```

Loops

```
for (int i=0; i<10; i++) { ... }

while (condition) { ... }

do { ... } while (condition);
```

goto — considered useful

In C, **goto** is legitimately used for error-handling cleanup at the end of a function — avoids nested if/else and duplicated cleanup code. Jump to a **failure**: label that frees resources and returns an error code.

```
buf = malloc(N);
if (!buf) goto failure;
f = fopen(name, "r");
if (!f) goto failure;
// ... do work ...
free(buf); return 0;
failure:
printf("Error %d\n", errno);
return 1;
```

8. Preprocessor Directives

Directive	Purpose	Example
#define NAME val	Replace NAME with val before compilation — constants and macros	#define BUF_SIZE (16) → use BUF_SIZE everywhere
#define MACRO(x) expr	Parameterised macro (text substitution, not a real function call)	#define DOUBLE(r) ((r)*2) — always parenthesise!
#include	Insert contents of a system header file	#include
#include "file"	Insert contents of a local header file	#include "point.h"
#ifdef / #ifndef / #endif	Conditionally compile blocks — useful for benchmarking or platform differences	#ifdef BENCHMARK ... #endif
#ifndef GUARD_H / #define GUARD_H / #endif	Include guard — prevents a header being included twice	Wrap every .h file with this pattern

Macros are text-substituted before compilation — they have no type safety. Always wrap macro arguments in parentheses to avoid operator-precedence bugs: #define DOUBLE(r) ((r)*2) not (r*2).

9. Header Files & Separate Compilation

Split code across files for maintainability. The header (.h) declares the public interface; the implementation (.c) defines it.

```
// point.h ← declarations only
#ifndef __POINT_H
#define __POINT_H
typedef struct { double x; double y; } point_t;
point_t* point_new(double x, double y);
void point_destroy(point_t* p);
void point_print(point_t* p);
#endif

// Compile: gcc -O0 -Wpedantic point_prog.c point.c -o point_prog
```

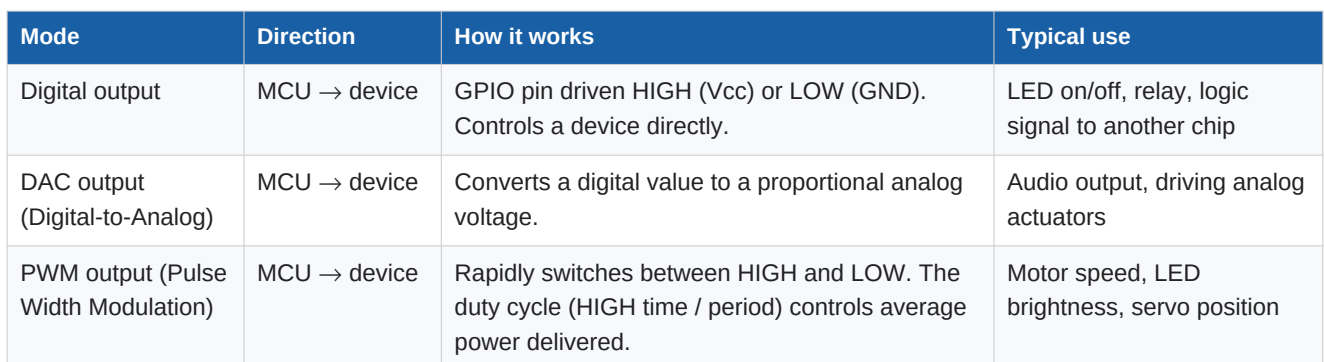
10. The Ten Commandments of C (summary)

#	Commandment
1	Run lint frequently and heed its warnings
2	Never follow a NULL pointer
3	Cast all function arguments to the expected type explicitly
4	Declare library function return types yourself if headers don't
5	Always check array (and string) bounds
6	Always check error codes — even when you think 'it can't happen'
7	Study the standard library; do not reinvent it without cause
8	Use consistent brace style to keep code readable for others
9	Keep external identifiers unique in the first six characters
10	Never assume a specific platform word size or byte order

Software Technology for Internet of Things | SDU MSc Software Engineering | Exam notes

Signal: the evolution of the value of some metric over time — a function $\text{time} \rightarrow \text{value}$.

Analog signals are continuous; all digital processing is discrete. The transition between them requires conversion:



Digital input	Device → MCU	GPIO pin read as HIGH or LOW based on threshold.	Button press, digital sensor flag
ADC input (Analog-to-Digital)	Device → MCU	Samples the analog voltage and converts to a digital integer value.	Temperature sensor, microphone, potentiometer

PWM waveform

HIGH ■■■■■■ ■■■■■■■■ ■■■■■■■■
 ■ ■ ■ ■ ■
 LOW ■■■■■■ ■■■■■ ■■■■
 ■←period→■←period→■
 ■←DC→■ (DC = duty cycle)

PWM does not produce a true analog signal — it produces a square wave. The device (motor, LED) integrates the power over time, responding to the average. A low-pass filter can smooth PWM into a near-analog voltage.

4. Serial Communication

Serial communication transfers data one bit at a time over a wire. Many protocols exist; the lecture focuses on RS232 and briefly mentions SPI and I2C.

Protocol	Pronunciation	Wires	Notes
RS232	R-S-two-three-two	3 minimum: TX, RX, GND	Simple, ubiquitous for MCU debug/serial console. The focus of this lecture.
SPI	"spy"	4: MOSI, MISO, SCLK, CS	Fast, synchronous, full-duplex. Good for sensors and displays.
I2C	"I-squared-C"	2: SDA, SCL	Multi-device on same bus using addresses. Slower than SPI.

RS232 frame format

Start bit	Data byte	Parity bit	Stop bit(s)
1 bit	7–8 bits	optional	1–3 bits

Each character is wrapped in a frame transmitted at a fixed **baud rate** (bits per second). The start bit signals the beginning of a frame; the stop bit(s) signal the end and give the receiver time to reset.

The optional **parity bit** provides simple error detection — it is set so the total number of 1-bits is either always even (even parity) or always odd (odd parity). A mismatch indicates a transmission error.

5. Handling Incoming Serial Data

When data arrives over serial, you must decide what to do with it. Two independent choices:

Data format choice

- **Machine-readable** (e.g. binary, fixed-width) — simple and fast to parse, compact
- **Human-readable** (e.g. ASCII text, JSON) — easier to debug with a terminal, larger payload

Dispatch mechanism choice

- **Lookup table** — array indexed by command byte; O(1)
- **Map** — key-value store; flexible but more overhead
- **Chain of branches** — if/else or switch; simple but O(n) worst case
- **Deterministic finite automaton (DFA)** — state machine; handles complex protocols efficiently

6. Deterministic Finite Automaton (DFA) for Serial Parsing

A DFA is a state machine with a finite set of states, an alphabet of inputs, a transition function (current state + input → next state), a start state, and a set of accept states.

For serial command parsing, each state represents progress through a recognised pattern. Each incoming byte triggers a transition. Reaching an accept state means a complete, valid command has been received.

Advantages over a chain of branches:

- Handles variable-length and multi-byte commands cleanly
- O(1) per byte processed — no re-scanning
- Easy to extend: add new states/transitions without restructuring existing logic
- Maps directly to enum + switch-case in C

Example skeleton in C

```
typedef enum { S_IDLE, S_CMD_A, S_CMD_B, S_DONE } state_t;

state_t state = S_IDLE;

void process_byte(char c) {
    switch (state) {
    case S_IDLE:
        if (c == 'A') state = S_CMD_A;
        else if (c == 'B') state = S_CMD_B;
        break;
    case S_CMD_A:
        // handle byte following 'A'
        state = S_DONE;
        break;
    case S_DONE:
        execute_command(); state = S_IDLE;
        break;
    }
}
```

The enum+switch pattern from Lecture 4 maps perfectly to a DFA: the enum defines states, the switch defines transitions. This is idiomatic C for embedded protocol parsing.

Lecture 6 — Sampling Physical Phenomena

Software Technology for Internet of Things | SDU MSc Software Engineering | Exam notes

1. General Data Collection Pipeline



The pipeline converts a continuous physical signal into a form that can be processed digitally and, if needed, reconstructed:

Stage	Description
Physical world / Transducer	A physical phenomenon (heat, light, force) is converted into a continuous electrical voltage by a transducer (e.g. thermistor, microphone).
Sampler (ADC)	The Analog-to-Digital Converter periodically captures the continuous signal and converts each snapshot into a discrete integer value.
Discrete signal	A time-series of (timestamp, value) pairs — the raw digital representation of the original analog signal.
Interpolator	Reconstructs an approximation of the original continuous signal from the discrete samples (e.g. linear interpolation, sinc interpolation).
Reconstructed signal	An approximation of the original; quality depends on sampling rate, bit depth, and interpolation method.

2. Sampling Theory

Sampling period T — the time between two consecutive samples.

Sampling frequency f_s — average number of samples per second.

$$f_s = 1 / T$$

Nyquist-Shannon Sampling Theorem: To guarantee perfect reconstruction of a signal, you must sample at least twice as fast as the highest frequency component in the signal:

$$f_s \geq 2 \times f_{max}$$

The minimum required rate ($2 \times f_{max}$) is called the **Nyquist rate**. Sampling below it causes **aliasing**.

Original signal (1 Hz sine): ~~~~

Sampled at 3 Hz ($> 2\times$): • • • • → correct reconstruction

Sampled at 0.8 Hz ($< 2\times$): • • → aliasing / wrong shape

Rule: $f_s \geq 2 \times f_{max}$ (Nyquist-Shannon theorem)

$T = 1 / f_s$ (sampling period)

Concept	Definition	Real-world example
Aliasing	A distortion caused by undersampling: different frequencies become indistinguishable, producing a false lower-frequency signal.	A spinning wheel filmed at too low a frame rate appears to spin backwards.
Moiré pattern	Aliasing in 2D images: interference between a regular pattern and the sensor's colour filter array.	Fabric photographed at certain scales shows shimmering colour bands.
Audio aliasing	Sampling audio below $2\times$ the highest audible frequency introduces phantom tones into the recording.	CD audio uses 44.1 kHz to capture up to 22.05 kHz (just above human hearing limit of ~20 kHz).

3. Timeseries & Resampling

Timeseries: a set of measurements mapping time to values — time ■ value.

At times the frequency of a timeseries must be adapted. This is called **resampling**:

- **Downsampling** — fewer samples represent the same signal (reduce data volume at the cost of detail).
- **Upsampling** — more samples generated from the same signal (increase apparent resolution, e.g. for synchronising two streams).

Errors introduced by resampling:

- Cutting off extreme values (peaks/troughs missed between samples)
- Removing fluctuations (rapid changes averaged away)

4. Analog-to-Digital Conversion — Sample & Hold Method

The most common ADC implementation uses a capacitor to capture the input voltage, then compares it against known reference levels to produce a binary output, one bit at a time:

1. Flush	Drain capacitor charge → connect to GND
2. Wait	Disconnect from GND; stabilise
3. Sample	Connect capacitor to input → charges up
4. Hold	Capacitor holds voltage = input voltage
5. Compare	Compare held charge to known reference bits

ADC design axis	Options	Trade-off
Comparison order	All-parallel (flash ADC) ← spectrum → all-sequential (SAR ADC)	Parallel: faster but more hardware (2^n comparators for n bits). Sequential: slower but minimal hardware.
Reference scale	Linear ADC — equal voltage steps between levels	Uniform precision across the full range.
Reference scale	Logarithmic ADC — smaller steps at low voltages, larger at high	Higher precision for small signals; matches human perception (e.g. audio).

5. Signal Imperfections

Noise & SNR

Noise sources:

- The sensor itself (thermal noise, shot noise)
- Electromagnetic interference picked up by analog wiring
- Quantisation error introduced by the ADC

The baseline noise level is the **noise floor**.

The **Signal-to-Noise Ratio (SNR)** measures what fraction of the signal is not noise. Higher = better.

Averaging improves SNR: averaging N samples doubles the signal (coherent addition) while many noise types partially cancel (incoherent), so SNR increases approximately by $\sqrt{N}$.

Outliers & Black Swans

Outlier: a sample that lies far from the bulk of the distribution.

Outlier removal discards these samples to avoid giving them undue weight in analysis. Risk: a removed sample may turn out to be a real, significant event.

Black swan: an event that models consider impossible because the data suggesting it was removed as an outlier. These occur in practice and often have serious consequences.

Examples: COVID-19 pandemic (2020), Fukushima nuclear accident (2011), 2008 financial crisis, 2001 dot-com bubble.

Removing outliers is a deliberate modelling choice with real consequences. Always document why samples were removed.

6. Sampling Bandwidth & Offloading

The raw bandwidth required to stream samples off a device grows with both sampling rate and bit depth (sample resolution):

Theoretical bandwidth = bit depth (bits) × sampling rate (samples/s)

Depth \ Rate	1 Hz	10 Hz	100 Hz	1 kHz	10 kHz	100 kHz
8 bit	8 bps	80 bps	800 bps	8 kbps	80 kbps	800 kbps
10 bit	10 bps	100 bps	1 kbps	10 kbps	100 kbps	1 Mbps
12 bit	12 bps	120 bps	1 kbps	12 kbps	120 kbps	1 Mbps
16 bit	16 bps	160 bps	1 kbps	16 kbps	160 kbps	1 Mbps

Effective bits vs noise floor

If the ADC has a noise floor of N bits, those N bits carry no usable information. Only (bit_depth – noise_floor) bits are meaningful:

Worked example (from lecture):

12-bit ADC, 2-bit noise floor, sampling at 8 kHz over UART.

Theoretical: (12 – 2) bits × 8,000 Hz = **80 kbps** of information.

Practical (UART overhead): UART frame = 1 start + 8 data + 0 parity + 2 stop = 11 bits per 8 data bits → 37.5% overhead.

80 kbps × 1.375 = **110 kbps** required baud rate.

Catch: if even one bit is corrupted on the serial line, the entire sample is wrong — the lecture notes this as an open problem. Solutions include checksums, error-correcting codes, or framing protocols.

Key insight: bit errors on a serial line are not uncommon, especially in electrically noisy environments. Streaming raw ADC samples without any error detection is fragile — always add a checksum or CRC in practice.

Lecture 7 — Radio Communication

Software Technology for Internet of Things | SDU MSc Software Engineering | Exam notes

1. Radio Frame Format

Data is transmitted as **frames**. Each frame has a header containing routing and control fields, a variable-length payload, and a CRC for error detection:

channel	type	source	dest.	size	payload	CRC
6 bit	4 bit	8 bit	8 bit	6 bit	0–26 B	8 bit

```
typedef struct frame_t {
    unsigned int channel : 6;
    unsigned int type : 4;
    unsigned int source : 8;
    unsigned int destination : 8;
    unsigned int size : 6;
    char* payload; // 64-bit pointer
    unsigned int crc : 8;
} __attribute__((__packed__)) frame_t;
// sizeof(frame_t) = (6+4+8+8+6+64+8)/8 = 13 bytes
```

`__attribute__((packed))` tells GCC not to insert alignment padding bytes between fields. Without it, `sizeof()` would be larger than the sum of bits due to word-alignment requirements.

2. Radio Spectrum & Frequency Tradeoffs

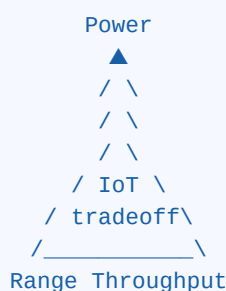
Radio waves occupy a **spectrum** of frequencies. Parts are regulated and reserved (TV, mobile, military); others are **open / unlicensed** (ISM bands) — IoT devices operate here.

Key frequency tradeoff (given same encoding and power):

- **Lower frequency** → longer range, lower throughput
- **Higher frequency** → shorter range, higher throughput

EM waves obey the **inverse square law**: signal intensity falls off proportional to $1/d^2$. This makes long-distance transmission infeasible without directional antennas or relay nodes.

3. The Three-Way Performance Tradeoff



The wireless performance space is defined by three dimensions that are in fundamental tension with each other:

- **Power** — energy consumed per unit time
- **Range** — maximum transmission distance
- **Throughput** — bits transferred per second

Improving any one dimension generally penalises at least one of the others. **True technological breakthroughs** that escape this tradeoff are rare.

This tradeoff is fundamental to choosing a radio standard for an IoT use case. There is no universal best — only the best fit for a specific application.

4. Signal Impairments

Impairment	Description	Consequence
------------	-------------	-------------

Noise floor	Baseline electromagnetic noise present even with no intentional signal. Every receiver has one.	A transmitted signal must be strong enough above the noise floor to be reliably decoded.
Bitflip	A single bit is received incorrectly because the SNR drops below the required threshold (plus safety margin).	Data corruption. Detected using CRC; mitigated by retransmission.
CRC (Cyclic Redundancy Check)	A checksum appended to every frame. The receiver recomputes the CRC and compares. More bits → higher detection coverage than a single parity bit.	Detects (but does not correct) most corruption. Never guarantees 100% detection.
Reflection	Signal bounces off surfaces (walls, floors, metal objects).	Causes multipath — copies of the signal arrive at different times, causing intersymbol interference.
Absorption	Signal energy is absorbed by materials (water, concrete, bodies).	Reduces received signal strength; shortens effective range.
Transparency	Some materials allow radio signals to pass through with low loss (glass, drywall at low frequencies).	Helps propagation indoors; varies strongly by frequency.
Constructive interference	Two signals add together in phase.	Increases received amplitude — beneficial but unpredictable.
Destructive interference	Two signals cancel each other out of phase.	Can completely null a signal at a specific location (dead spot).
Multipath	Multiple copies of the same signal arrive via different paths.	Combines constructive and destructive interference; causes fading and inter-symbol interference.
Collision	Two or more packets overlap in both time and space.	Signals combine destructively; typically both are lost at that location.

5. Network Topologies

Line A — B — C — D	Star <pre> A /\ B C D </pre>	Ring <pre> A — B D — C </pre>
Tree <pre> A /\ B C /\ D E </pre>	Tiered <pre> [L1] [L2 L2] [L3 L3 L3] </pre>	Mesh <pre> A—B—C X X D—E—F </pre>

Topology	Characteristics	Typical IoT use
Line	Simple chain; single point of failure per link	Sensor chains along a corridor
Star	Central hub; all communication via hub	WiFi access point, Zigbee coordinator
Ring	Each node relays to next; failure isolates segment	Industrial fieldbus (rare in IoT)
Tree	Hierarchical; combines star clusters	Building automation with floor gateways
Tiered	Multiple layers (device → edge → fog → cloud)	Typical IoT deployment architecture
Mesh	Every node can relay; self-healing, redundant paths	Zigbee/Thread smart home, industrial IoT

6. OSI Model (Extended to Level 10)

The OSI model describes communication as a stack of layers. The lecture extends the classic 7-layer model to 10 levels, adding the human and organisational context above the application layer:

L#	Layer	PDU	Responsibility
10	Government	"legislation"	Rules for organisations and compliance
9	Organization	"guideline"	Organisational framework
8	Individual	"correspondence"	Application user
7	Application	data	APIs
6	Presentation	data	Convenient translations (e.g. encryption, coding)
5	Session	data	Session management of connections
4	Transport	segment / datagram	Connection-orientation, multiplexing, flow control
3	Network	packet	Multi-node routing
2	Data Link	frame	Reliable* delivery of frame to local named receiver
1	Physical	bit (or symbol)	Electrical, audio, or radio link

Layers 1–4 are the most relevant for embedded IoT. L1 (Physical) handles the radio; L2 (Data Link) handles framing and local delivery; L3 (Network) handles routing between nodes; L4 (Transport) handles end-to-end reliability. Layers 8–10 are the lecture's whimsical addition for context.

7. Message Brokering & Gateways

A **gateway** bridges two networks that use different protocols or frequency bands. It terminates at level N of one network stack and re-transmits at level M of another, translating between the two.

Example: a smart-home hub that receives Zigbee (L2) frames from sensors and forwards the data over WiFi (L3/L4) to a cloud server. The hub must understand both protocol stacks.

This is necessary whenever:

- Two networks use incompatible physical layers (different frequencies)
- Two networks use incompatible addressing or framing schemes
- Data must be re-encoded, filtered, or aggregated before forwarding

8. Acknowledgements (ACKs) & Retransmission

ACK strategy	When ACK is sent	Trade-off
Per-frame ACK	Immediately after each frame is received	Fast error detection; high overhead for large multi-frame packets
Per-packet ACK	After all frames of a packet are received	Lower overhead; entire packet retransmitted if any frame lost
Piggyback ACK	When data happens to flow in the reverse direction anyway	Zero extra overhead; ACK may be delayed if reverse traffic is infrequent

Frames are often dropped due to detected CRC corruption or complete loss (no signal). Without ACKs the sender has no way of knowing whether a frame was received. ACK + retransmit adds reliability at the cost of latency and radio budget.

9. CSMA & Example Radio Stack (TinyOS CC2420)

CSMA — Carrier-Sense Multiple Access: before transmitting, a node listens to check if the medium is already in use. If busy, it waits (with a random back-off). This reduces — but does not eliminate — collisions on a shared wireless medium.

Variants: **CSMA/CA** (Collision Avoidance — used in WiFi and Zigbee) adds a random back-off before transmitting even when the channel appears free. **CSMA/CD** (Collision Detection) detects collisions mid-transmit and aborts — used in wired Ethernet but impractical for wireless.

The TinyOS CC2420 radio stack illustrates how multiple protocol concerns are layered as composable software components (TEP 126):

Layer	Dir.	Responsibility
Active Message	TX/RX	Port-like addressing — dispatch to different handlers by message type
Unique Send	TX	Assigns unique packet IDs to outgoing packets
Packet Link	TX	Automatic retransmission until ACK received
Low-Power Listening	TX/RX	Duty-cycles the radio — periodic wake to listen, sleep otherwise
Unique Receive	RX	Filters duplicate packets (e.g. retransmitted copies)
TinyOS Network	TX/RX	Interoperability across different TinyOS network stacks
CSMA	TX/RX	Carrier-Sense Multiple Access — listen before transmit to avoid collisions

Low-Power Listening (LPL) is particularly important for battery-powered IoT nodes — it allows the radio to sleep most of the time, waking periodically to check for a preamble. A sender must transmit a long preamble to ensure the receiver wakes during its window.

Lecture 8 — Radio Standards

Software Technology for Internet of Things | SDU MSc Software Engineering | Exam notes

This lecture was a student-driven presentation. The topics below summarise all standards and technologies covered by the student groups.

1. Radio Standards — Master Comparison Table

Standard	Freq.	Range	Throughput	Power	Topology	Max nodes	Use case
WiFi (802.11)	2.4/5/6 GHz	~50–100m	High (Mbps)	High	Star (AP)	~200 practical	LAN, streaming, web
BLE	2.4 GHz	~10–100m	Low–medium	Very low	Star/Mesh	7–20 (mesh: 37k+)	Sensors, wearables
ESP-NOW	2.4 GHz	~100m	Low	Very low	P2P, broadcast	<20 (<10 encrypted)	ESP32 peer-to-peer
ESP-WIFI-MESH	2.4 GHz	Extended	Medium	Medium	Mesh	Hundreds	ESP32 mesh coverage
Zigbee	2.4 GHz / 868–915 MHz	10–80m	250 kbps	Low	Mesh	65,000	Smart home, building
Z-Wave	868/908 MHz	50–100m	100 kbps	Low	Mesh	232	Smart home security
Thread	2.4 GHz	10–30m (mesh)	250 kbps	Low	IPv6 Mesh	16,000	Smart home (Matter)
Matter	Any (WiFi/Thread/Eth)	Depends on L1	Depends on L1	Depends on L1	Any	2^64 (IPv6)	Unified consumer IoT
LoRa / LoRaWAN	Sub-GHz ISM	2–15 km (urban–rural)	0.3–50 kbps	Ultra-low	Star-of-stars	Millions (server-side)	LPWAN: agriculture, city
Sigfox	868/902 MHz	10–50 km	100 bps (UL) / 600 bps	Ultra-low	Star (operator)	Millions	Ultra-low-data LPWAN
NB-IoT	Licensed LTE bands	Nationwide + deep indoor	20–250 kbps	Ultra-low	Cellular (operator)	Millions	Smart metering, tracking
LTE-M	Licensed LTE bands	Nationwide	Up to 1 Mbps	Low	Cellular (operator)	Millions	Mobile IoT, voice+data
4G LTE	Licensed	Nationwide	Up to 150 Mbps	Very high	Cellular (operator)	Millions	Broadband, video
5G	Licensed (mmWave+sub-6)	Varies (m to km)	Up to 10+ Gbps	Very high (mmWave)	Cellular	Millions+	Critical IoT, AR/VR, auto
NFC	13.56 MHz	<4 cm	~400 kbps	Passive possible	P2P / tag-reader	1:1	Payments, access, pairing
RFID	125kHz / 13.56MHz / UHF	cm to ~10m	Low	Passive / active	Reader-tag	Depends on reader	Inventory, access control
Wireless HART	2.4 GHz	~100m (mesh)	250 kbps	Low–medium	Mesh	~100	Industrial process control
802.15.4 / 6LoWPAN	2.4 GHz / 868–915 MHz	10–100m	250 kbps	Low	Star / Mesh	65,535	Base for Zigbee, Thread

2. Key Radio Technology Concepts

Spread Spectrum & Frequency Hopping

Spread spectrum distributes a signal over a wider bandwidth than needed, making it more robust against interference, jamming, and noise. Invented by Hedy Lamarr and George Antheil (inspired by 88 piano keys).

FHSS (Frequency-Hopping Spread Spectrum) — transmitter and receiver stay in sync and hop between frequencies in a coordinated pattern. Both BLE and Zigbee use FHSS.

- **Pros:** interference resistance, jamming resistance, privacy
- **Cons:** complexity, tight synchronisation required, uses more bandwidth

Beamforming

Uses multiple antennas with controlled phase delays to direct signal energy in a specific direction via constructive and destructive interference. Used in WiFi (802.11ac/ax) and 5G.

- **Pros:** longer range, less interference to others, better SNR
- **Cons:** precise placement needed, hardware cost, limited by obstacles

Backscatter

Transmits data by modulating how an incoming RF signal is reflected — no transmitter power required. The tag changes its antenna impedance to encode data in the reflected signal.

- **Pros:** ultra-low power (possibly passive)
- **Cons:** very short range, requires external RF source

Software-Defined Radio (SDR)

Implements radio functions in software (DSP) rather than fixed hardware circuits. Can change frequency, modulation, and protocol by reprogramming.

- Operates at OSI Layer 1
- Ideal for research, protocol experiments, and rapid prototyping
- Cost: €20–50 (RTL-SDR) to €1000+ (research grade)
- **Cons:** needs high CPU, lower performance than dedicated hardware, legal restrictions on frequencies

Low-Power Listening (LPL)

MAC-layer technique: nodes sleep by default and wake periodically to sample the channel (Clear Channel Assessment, CCA). If activity detected, stay awake; otherwise sleep again.

- **CCA thresholds:** Signal Detect (preamble?) and Energy Detect (anyone transmitting?)
- **Benefit:** drastically reduces idle listening power
- **Cost:** sender must transmit a long preamble; adds latency; false wake-ups from noise

3. Mesh Networking

Mesh network: every node can act as both endpoint and relay. Partial mesh is the practical IoT reality — nodes connect to some neighbours and route through them to reach others.

Key property: no single point of failure for routing — if one node dies, traffic re-routes around it.

Good when: large coverage area with poor infrastructure (warehouses, mines, disaster zones), dense deployments, need for self-organisation.

Cost of mesh	Explanation		
Latency	Every hop adds propagation + processing delay. 5 hops = 5× direct-link latency.		
Energy	Relay nodes burn more power than pure endpoints — bad for battery-powered sensors.		
Complexity	Routing logic runs on every node; non-trivial on RAM-constrained MCUs.		
Routing overhead	Routing table maintenance consumes bandwidth, especially during topology changes.		
Throughput degradation	Nodes share the radio channel — each relay hop roughly halves usable throughput.		

Routing strategy	Mechanism	Pros	Cons
Proactive	Continuously computes and stores routes to all nodes	Routes immediately available	High overhead; impractical for large/dynamic nets
Reactive (on-demand)	Discovers routes only when a transmission is needed	Saves bandwidth	Route discovery adds latency

Hybrid	Proactive on static backbone; reactive for mobile nodes	Balanced performance	More complex to implement
--------	--	----------------------	---------------------------

Routing metric	Core focus	Key advantage	Limitation
Hop Count	Shortest path	Simple, fast	Ignores link quality
ETX	Packet loss + path length	Avoids lossy links	Ignores transmission rates
ETT	Link capacity/speed	Accounts for varying rates	Ignores channel interference
WCETT	Intra-flow interference	Promotes channel diversity	No loop-free guarantee
MIC	Full interference (intra + inter)	Most accurate	Highly complex

Distance Vector Routing & Counting to Infinity

Each node maintains a table: to reach destination X, go via neighbour Y at cost Z. Nodes share tables with direct neighbours; updates propagate using the **Bellman-Ford algorithm** (cost = neighbour cost + link cost).

Counting to infinity problem: when a link breaks, nodes can form a routing loop incrementing the cost forever.

Solutions:

- **Max hop count** — declare unreachable above a threshold
- **Split horizon** — don't advertise a route back to the neighbour you learnt it from
- **Poisoned reverse** — advertise the route back with infinite metric to explicitly mark it unreachable

4. Eventual Consistency & Vector Clocks

Eventual consistency

A consistency model: if no new updates are made, all replicas will *eventually* have the same state. Temporary inconsistencies are allowed; the system converges over time.

CAP theorem context: choosing Availability + Partition Tolerance over strong Consistency.

- **Advantages:** high availability, better performance, scales well
- **Disadvantages:** reads may return stale data, conflict resolution needed, harder to reason about correctness

Vector clocks

A list of per-node logical counters used to track causality in distributed systems — detects which update happened before another and identifies concurrent (conflicting) updates.

Rules: increment own counter on update; send vector with message; merge on receive (take element-wise maximum).

Comparison: $A < B$ if all A's counters $\leq$ B's (A happened before). Neither $A < B$ nor $B < A \rightarrow$ concurrent conflict.

- **Limitations:** size grows with node count; metadata overhead per message; detects conflicts but does not resolve them

5. Over-The-Air (OTA) Updates

OTA allows IoT devices to receive firmware updates remotely without physical access. The device downloads new firmware over WiFi/Ethernet/Bluetooth while running, then reboots into it.

ESP-IDF implementation: two firmware partitions (active + inactive). New firmware is written to the inactive partition, verified, and the boot partition pointer is switched. On failure, optional rollback returns to the previous partition.

Requirements: OTA-supporting partition table, network connection, firmware hosted on a server, OTA logic in the application. Optionally: rollback support and secure HTTPS delivery.

6. Network Time Protocol (NTP)

NTP synchronises device clocks to a reference time server using a hierarchy of time sources (stratum 0 = atomic clock, stratum 1 = directly connected, stratum 2 = synced to stratum 1, ...).

NTP measures the round-trip delay between client and server, estimates one-way propagation, and adjusts the local clock accordingly. Accuracy: typically 1–50 ms over the internet; sub-ms on a LAN.

Why it matters for IoT: without time sync, timestamps from different devices cannot be reliably compared or correlated, making multi-stream analysis unreliable. See also: vector clocks for logical ordering when NTP is unavailable.

Tradeoff: NTP requires a network connection and trusted time server; adds a dependency and potential privacy exposure.

7. WiFi (IEEE 802.11) Deep Dive

Aspect	Detail
Frequencies	2.4 GHz (11–14 ch, 3 non-overlapping), 5 GHz (25 non-overlapping), 6 GHz (59 non-overlapping)
Channels	Sub-divisions of the spectrum to reduce interference between neighbouring APs
SSID	Human-readable network name. Devices reconnect to any AP sharing the same SSID (enables roaming)
Eduroam	Uses shared SSID + RADIUS authentication servers — device authenticates with its home institution's RADIUS, enabling worldwide access
Joining a network	1. Beacon/Probe discovery → 2. Authentication handshake → 3. Association request/response → 4. 4-way handshake (for encrypted networks)
4-way handshake	Derives session keys from shared password (PMK) + random nonces (ANonce, SNonce) → PTK for unicast, GTK for multicast. Prevents eavesdropping.
OSI layers	L1: 2.4/5/6 GHz radio. L2: subdivides into MAC (medium access) + LLC (logical link control). Uses Ethernet-style frames with preamble, MAC addresses, type, payload (46–1500 B), CRC.
Max devices	Hardware and subnet limited; private IP ranges give up to ~17.9M addresses theoretically

8. BLE (Bluetooth Low Energy) Deep Dive

Aspect	Detail
vs Classic Bluetooth	BLE: very low power, intermittent connection, sensors/IoT. Classic: high power, continuous, audio/streaming
Channels	40 total: 3 advertising (discovery) + 37 data channels. Classic uses 79 channels.
Peak power	10–15 mA (BLE) vs 30 mA (Classic)
GAP (Generic Access Profiles)	Defines how devices advertise, scan, and connect. Roles: Central (e.g. phone) and Peripheral (e.g. fitness tracker)
GATT (Generic Attribute Profiles)	API for data exchange. Server provides data; Client reads/writes. Structure: Service → Characteristic → Descriptor. Operations: Read, Write, Notify
GATT example	Heart Rate Service → Heart Rate Characteristic (72 BPM) → Phone subscribes → gets live updates
Network types	1:1, broadcast, pub/sub, mesh — supporting from 1 to theoretically 37,767 connections

9. LPWAN Standards (LoRa, Sigfox, NB-IoT, LTE-M)

Low-Power Wide Area Networks — the right choice when devices need to send small amounts of data infrequently over long distances on a battery.

Standard	Operator	Coverage	Data rate	Key traits
LoRa / LoRaWAN	Private (TTN community) or commercial	2–15 km; LEO satellite deployments expanding	0.3–50 kbps	Chirp Spread Spectrum modulation; spreading factor trades range vs throughput; star-of-stars topology via gateways to Network Server
Sigfox	Proprietary operator (went bankrupt Feb 2022; network acquired/continued in some regions)	10–50 km	100 bps UL / 600 bps DL	Ultra-narrow band; very restrictive (140 msg/day UL, 4 DL); extremely low power; operator dependency is a risk
NB-IoT	Mobile network operators (licensed spectrum)	Nationwide + excellent deep indoor penetration	20–250 kbps	Built into LTE bands; SIM required; ideal for stationary devices that need guaranteed delivery (meters, alarms)
LTE-M	Mobile network operators	Nationwide	Up to 1 Mbps	Supports voice + data + mobility; lower power than standard LTE; suits mobile IoT (asset tracking, wearables)

LoRa spreading factor: higher SF = longer range, lower throughput, more airtime. LoRaWAN duty-cycle limits (<1% in EU ISM band) mean a node can only transmit a few seconds per hour — design data payloads accordingly.

10. Cellular Standards (GSM / 4G LTE / 5G)

Standard	Power	Data rate	Range/penetration	Best for
GSM (2G)	High (peaks)	Very low	Moderate	Legacy voice/SMS
LTE (4G Cat4)	Very high	Up to 150 Mbps	Moderate	Broadband, real-time video
LTE-M	Low	Up to 1 Mbps	High	Mobile IoT, voice + data
NB-IoT	Ultra-low	20–250 kbps	Excellent (deep indoor)	Stationary sensors, meters
5G sub-6 GHz	High	Up to 1 Gbps	Good	High-density IoT, AR, auto
5G mmWave	Very high	Up to 10+ Gbps	Poor (short range, walls block)	Ultra-high throughput, fixed wireless

5G key differences from 4G: network slicing (virtual dedicated networks), ultra-low latency (<1 ms), massive MIMO, edge computing capability. Logic can be placed at the edge (MEC — Multi-Access Edge Computing) rather than centralised cloud.

11. Short-Range & Industrial Standards

Standard	Frequency	Range	Through put	Key traits
NFC	13.56 MHz	<4 cm	~400 kbps	Passive (no battery in tag) or active. Subset of RFID. Uses: payments, access cards, device pairing.
RFID	125kHz / 13.56MHz / UHF	cm to ~10m	Low	Passive or active tags. Reader-initiated. Uses: inventory, supply chain, access control.

WirelessHART	2.4 GHz (IEEE 802.15.4)	~100m (mesh)	250 kbps	Designed for industrial process control. Time-synchronized mesh. Coexists with existing wired HART infrastructure. Very reliable, deterministic latency.
802.15.4	2.4 GHz / 868–915 MHz	10–100m	250 kbps	Base physical + MAC layer used by Zigbee, Thread, 6LoWPAN, WirelessHART.
6LoWPAN	(over 802.15.4)	Same as 802.15.4	Same	IPv6 adaptation layer for 802.15.4 — compresses IPv6 headers to fit in small 802.15.4 frames (max 127 bytes). Enables IP-native IoT.

12. OSI Layer Placement Summary

Standard	L1 Physical	L2 Data Link	L3 Network	L4+ Notes
WiFi	2.4/5/6 GHz	MAC + LLC (802.11)	IP (Ethernet frames)	TCP/UDP; HTTPS
Zigbee	2.4 GHz / 868 MHz	IEEE 802.15.4 MAC	Zigbee mesh	Zigbee device objects at application
Z-Wave	868/908 MHz	Z-Wave MAC/LLC	Z-Wave mesh	Z-Wave commands
Thread	2.4 GHz (802.15.4)	IEEE 802.15.4	IPv6	TCP/UDP; Matter app layer
Matter	Any (WiFi/Thread/Eth)	Any IPv6-capable	IPv6	Peer-to-peer; E2E encryption
BLE	2.4 GHz (FHSS)	BLE MAC	—	GAP (connection), GATT (data API)
LoRaWAN	Sub-GHz chirp	LoRa MAC	LoRaWAN Network Server	Application layer above NS
NB-IoT / LTE-M	Licensed LTE	LTE MAC	IP (SIM/operator)	TCP/UDP/CoAP/MQTT
6LoWPAN	802.15.4	802.15.4 MAC	IPv6 (compressed)	UDP/CoAP typical

Lecture 9 — Stream Processing

Software Technology for Internet of Things | SDU MSc Software Engineering | Exam notes

1. Anatomy of a Typical IoT Cloud

A typical IoT cloud has these key structural properties:

- **Large number of incoming streams** — from many devices across many customers
- **Privacy isolation** — customers expect their data to remain private (legal obligation); processing setups are approximately identical across customers
- **Multiple processing justifications** — promoted product functionality, real-world product evaluation, pattern detection for future features, and health monitoring of the device fleet
- **Asymmetry** — potentially large number of incoming streams, but only a small number of processed results are shown to the customer
- **Potentially large number of outgoing streams** — to dashboards, alerting systems, third-party integrations

2. Processing Graphs & Stream Nodes

A stream processing system is modelled as a directed graph of nodes. Each node is an independent piece of logic. The graph is also called a **processing graph** or **DAG (Directed Acyclic Graph)**.



Stream process node — an independent piece of logic with:

- **State** — private mutable data the node maintains between firings
- **Zero or more input streams** — data flowing into the node
- **Zero or more output streams** — data flowing out

Two degenerate cases:

- **Source** — produces output only (e.g. a sensor reading injector, a database query, a timer)
- **Sink** — consumes input only (e.g. a database writer, a dashboard updater, an alerting endpoint)

3. Node Design Properties

Property	Question	Options / Notes
Overload policy	What happens when data arrives faster than the node can process it?	Buffer (queue it up — guarantees completeness, increases latency) or Drop (discard excess — preserves timeliness, loses data). COP Theorem: you cannot guarantee both completeness and timeliness under overload.
Consumability	Is data consumed (removed) after processing, or retained for other consumers?	Consumed: each message processed by exactly one node instance (queue semantics). Retained: multiple consumers can read the same data (log/topic semantics, e.g. Kafka).
Trigger condition	For a multi-input node, when does it fire?	Barrier sync : wait for all inputs to have a new value (strict alignment). Latest value : fire when any input arrives, using the most recent value of all others. Choice affects what happens when one stream is slow or drops values.
Trigger arbitrariness	Can a node fire without any inputs arriving?	Yes — enables timeouts, heartbeats, periodic aggregation, and watchdog behaviour.

Batch size	Should the node operate on one item at a time or accumulate a batch first?	Larger batches improve throughput (amortise overhead) but increase latency. Small or 1-item batches minimise latency but have higher per-item overhead.
Fault tolerance	What is the right policy when a node crashes?	Restart (possibly losing in-flight state), checkpoint and replay, or idempotent design so reprocessing is safe.

COP Theorem (Completeness–Or–Timeliness):

When a stream processing node is overloaded — data is produced faster than it can be consumed — you must choose one guarantee:

- **Completeness:** every message is eventually processed (buffer it) — but latency grows unboundedly if overload persists
- **Timeliness:** results are always current (drop old/excess messages) — but some data is permanently lost

There is no free lunch. The right choice depends on the use case: a billing system needs completeness; a live dashboard needs timeliness.

4. Concurrency Concerns & Remedies

Concern	Problem	Remedy
Context switching	Switching the OS from one process/thread context to another is one of the most expensive operations that developers rarely account for. Even preempting one thread with another is costly.	Use multiplexing (cooperative scheduling within a single thread) instead of OS-level threads where possible. BEAM's scheduler uses this approach — lightweight processes multiplexed over OS threads.
Side effects	Code that reads or writes shared state (memory, files, network) generates cache misses and requires synchronisation — the root cause of most concurrency bugs.	Design nodes as pure functions (output depends only on input). Without side effects, concurrent execution is trivial — no locks needed. This is why functional programming (Elixir) is used for cloud-tier IoT.
Network / shared storage	In shared architectures, every update may require coordination across machines. Disk access is ~25 million times slower than a CPU register.	Use a share-nothing architecture — each update is satisfied by a single node, avoiding distributed coordination. Quote: "Going to disk is 25 million times slower than hitting a general purpose register. Design accordingly." — Robert Love, GUADEC 2005

5. Message Queues

Message queues decouple the producer of data from its consumers, enabling asynchronous processing and absorbing bursts of incoming data. They are the right tool when **all data must be processed** regardless of the availability of downstream components.

Data ingestion pipeline use case

Incoming data must pass through categorisation, validation, and sanity checking before reaching the database. A failed step can be retried using a **TTL (Time To Live)** — messages are re-queued up to a maximum number of attempts before being dead-lettered.


```
Incoming data  
↓ [Message Queue]  
Categorization  
↓ [Message Queue]  
Validation  
↓ [Message Queue + TTL/retry]  
Sanity Check 1 ■■  
Sanity Check 2 ■■  
↓ ■ (parallel sanity workers)  
Database ■  
←■■■■■■■■■■■■■■■■
```

Queue property	Description
Decoupling	Producer and consumer are independent — the producer does not need to know which consumers exist or whether they are currently running
Buffering	The queue absorbs bursts — a slow consumer won't cause the producer to block or drop data
TTL / retry	Failed messages can be re-queued with a decrementing TTL; expired messages go to a dead-letter queue for manual inspection
At-most-once / at-least-once / exactly-once	Different delivery guarantees with different tradeoffs in performance and complexity
Durability	Persistent queues survive crashes; in-memory queues are faster but lose data on restart

6. Event Buses

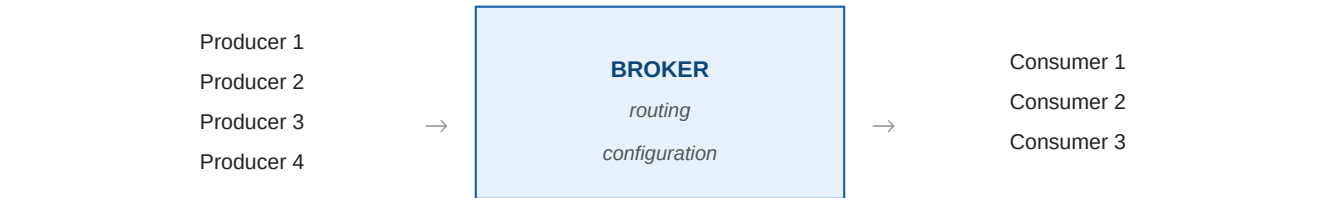
Event buses are suited for time-sensitive, low-latency stream processing where **timeliness matters more than completeness**. Unlike message queues, event buses typically do not guarantee delivery to every consumer and do not retry.

- **Use case:** a stream of timestamps arrives and must be adjusted for timezone before being forwarded to consumers. Timeliness is critical — a stale timestamp is worse than no timestamp.
- **Design implication:** use an event bus (not a queue) when:
 - The data has a short useful lifetime (real-time dashboards, alerts)
 - A slow consumer should not hold up fast consumers
 - Occasional data loss is acceptable in exchange for lower latency

Design implication: use an event bus (not a queue) when:

- The data has a short useful lifetime (real-time dashboards, alerts)
- A slow consumer should not hold up fast consumers
- Occasional data loss is acceptable in exchange for lower latency

7. Publish-Subscribe (Pub/Sub) Pattern



Publish-subscribe is a communication pattern in which:

- Messages are **published** to a named topic on a **broker**
- Clients **subscribe** to topic names — they do not need to know who publishes
- When a message is published, the broker **routes** it to all matching subscribers

The broker holds a **routing table** (subscriptions) and a **configuration** (topic definitions, QoS settings).

- Messages are **published** to a named topic on a **broker**
- Clients **subscribe** to topic names — they do not need to know who publishes
- When a message is published, the broker **routes** it to all matching subscribers

The broker holds a **routing table** (subscriptions) and a **configuration** (topic definitions, QoS settings).

System	Intelligence	Subscription model	Notes
--------	--------------	--------------------	-------

Kafka	Dumb broker / smart client	Subscribe to exact topic name	Client handles all filtering, aggregation, and offset management. High throughput, log retention, replay. KSQL/Camel add query layer on top.
MQTT	Smart broker / dumb client	Subscribe using topic pattern with + and # wildcards	Broker routes by pattern matching. Simple clients. Very popular in IoT. QoS 0/1/2. Last will and testament. Persistent sessions for offline buffering.
sMAP	Smart broker / dumb client	Subscribe using query over per-stream key-value metadata store	Query selects streams by metadata properties. More powerful than pattern matching.
"Data Portal"	Smart broker / dumb client	Subscribe using query over ontological or graph model	Most expressive — subscribe using SPARQL/Cypher-style queries over a full information model (e.g. Brick schema). Highest abstraction; highest broker complexity.

8. MQTT Deep Dive

Feature	Detail
Topic structure	Hierarchical strings with '/' as separator. Example: building/floor3/room12/temperature
'+' wildcard	Matches exactly one topic level. building/+ /room12/temperature matches any floor.
'#' wildcard	Matches zero or more levels (must be last). building/floor3/# matches all streams on floor 3.
QoS 0 — At most once	Fire and forget. No ACK, no retry. Fastest, least reliable.
QoS 1 — At least once	ACK required. Retransmit until ACK received. May deliver duplicates — receiver must be idempotent.
QoS 2 — Exactly once	4-way handshake ensures exactly one delivery. Slowest, most reliable.
Last will and testament	A message the broker publishes on behalf of a client when it disconnects unexpectedly. Useful for presence detection.
Persistent sessions	Broker buffers QoS 1/2 messages for a client while it is offline. Client reconnects and receives missed messages.

MQTT is deliberately simple — it is designed to run on constrained devices with limited memory and intermittent connectivity. The broker does the heavy lifting; clients stay lean.

9. Distributed System Failures

Rule: "As the number of machines in a system goes towards infinity, the time to one of them breaking goes to zero."

As a system grows, failure handling becomes a first-class concern, not an afterthought. Support systems address this at the service level:

- Run a service on **n replicas** — tolerate up to n–1 failures
- **Auto-restart** failed services
- **Distributed filesystem** co-located with compute (e.g. HDFS/Ceph)
- **Geographic redundancy** — survive entire data-centre failures

Technology	Granularity	Best for	Notes
------------	-------------	----------	-------

BEAM / Erlang OTP	Process (very fine — millions per node)	Homogeneous, high-concurrency, low-latency services	Supervision trees; hot code updates; actors restart in microseconds. Ideal for the IoT cloud/fog tier.
Kubernetes	Container (coarse — seconds to restart)	Heterogeneous microservices in diverse languages	Declarative desired state; auto-scaling; service mesh. Higher restart cost than BEAM.
Docker Swarm	Container (similar to Kubernetes)	Simpler deployments where full Kubernetes is overkill	Lighter orchestration; less ecosystem.
Ceph	Storage (object, block, file)	Distributed filesystem co-located with services	Often paired with Kubernetes for persistent storage.

BEAM and Kubernetes are often combined: BEAM handles high-granularity in-process fault tolerance for the core IoT processing logic; Kubernetes provides infrastructure-level orchestration and scaling.

Lecture 10 — Performance Evaluation

Software Technology for Internet of Things | SDU MSc Software Engineering | Exam notes

1. Performance Evaluation in the Development Lifecycle

Performance evaluation is not a one-time activity at the end of a project — it is a **feedback loop** integrated into the development cycle:



Performance evaluation feeds results back into **Requirements** and **Analysis**. A result that violates a requirement triggers a new design iteration. The loop never truly closes — it keeps the system aligned with non-functional requirements over time.

System definition: a collection of (mostly) independent components combined in a particular configuration to solve a problem. Some components (e.g. IoT devices) may be unavailable during evaluation and must be replaced with **stubs**. Often only a subsystem or single component is under evaluation.

2. Experimental Computer Science

"Science classifies knowledge. Experimental science classifies knowledge derived from observations. The experimenters construct apparatus to measure a given phenomenon; they then attempt to collect data sufficient to support their hypotheses about that phenomenon."

— Peter J. Denning

This is the intellectual framework for making performance arguments. It is not IoT-specific — it applies to any system where non-functional properties must be argued with evidence.

3. Core Terminology

Term	Definition	Example
Hypothesis	A testable expectation about system behaviour, usually rooted in non-functional requirements and intuitions.	"The system throughput will gracefully decline as concurrent connections exceed the number of physical CPU cores."
Apparatus	The system under evaluation — constructed to fit the requirements, not to fit the experiment. May require instrumentation to allow observation.	The IoT gateway plus stub sensors and a load generator.
Test	A designed scenario that highlights a relevant performance metric. Can be manual, partially automated, or fully automated. Repetition is key — prefer automation.	Send 1000 sensor readings/s and measure database insert latency.
Repeatability	For a conclusion to be trustworthy it must be reproducible: thorough documentation + enough repetitions to characterise the distribution of results.	Running the same workload 30 times to get a stable latency distribution.
Goal	The first step — defines what you are trying to learn. All subsequent choices serve these goals. Avoid bias ('show A is better than B').	"Determine the maximum sustained throughput of the ingestion pipeline."
Service	A specific operation the system provides, used as the unit of measurement.	Route a packet; insert a sensor reading; respond to a sensor event.

Metric	A criterion for quantifying and comparing performance. Always has a unit. Distinct from parameters.	Delay [ms], memory consumption [MiB], throughput [samples/s].
Parameter	A configuration value of the system. Kept fixed during a run unless it is being varied as a factor. Must be documented.	Buffer size, thread count, timeout, number of retries.
Factor	A parameter that is deliberately varied across experiments to study its effect on a metric. Assigned a set of discrete values.	Thread count $\in \{1, 2, 4, 8, 16\}$.
Workload	A predefined sequence of inputs fed to the system at predetermined times. Can be synthetic (repeating a request class at a frequency) or a replay of historical recordings.	Send INSERT requests at 100 req/s for 60 seconds.

4. Experimental Design

An experimental design is a construct that enables systematic experimentation with maximum value at minimum effort. It must:

Requirement	Details
1. Include necessary stubs	Replace unavailable components (e.g. real sensors) with stubs that simulate their behaviour.
2. Fix all parameters	Operate with known, documented values for every configuration parameter not being varied.
3. Stub all dependencies	Replace external services with controlled stubs so results are not affected by external variability.
4. Produce the workload	Implement or configure a workload generator that feeds inputs to the system at the required rate and pattern.
5. Iterate over factors and workloads	Systematically vary each factor across its value set and repeat the experiment for each combination.
6. Instrument relevant components	Add logging, counters, and probes to capture metric values during execution. Output to log files.
7. Do not influence the system	The measurement apparatus must not significantly affect the system's performance (e.g. a slow logger should not throttle throughput).

A full-factorial experiment varies all combinations of all factor values. This becomes expensive quickly. Alternatives: hill-climbing (tune one factor at a time), fractional factorial designs, or Latin hypercube sampling.

5. Complete Evaluation Methodology

Factors



Parameter Mapping ■■→ Testcase Generator



Workload Generator



Implementation (System Under Test)

■ (via test harness)



Instrumentation / Logfiles



Analysis ■■→ Visualization

Step	What happens
Factors → Parameter Mapping	Each factor value is translated into concrete system configuration parameters (e.g. factor 'thread count = 4' sets env var WORKERS=4).
Parameter Mapping → Testcase Generator	The testcase generator produces a list of configurations to run, one per combination of factor values.
Testcase Generator → Workload Generator	The workload generator is configured for each testcase and produces the input stream for that run.
Workload Generator → Implementation	The workload is fed into the system under test via a test harness.
Implementation → Instrumentation → Logfiles	The system's instrumentation records metric values during execution and writes them to log files (one or more per component/machine).
Logfiles → Analysis → Visualization	Log files are parsed, events identified, metrics extracted, and results visualised to support conclusions against the goals.

6. Time Measurement Challenges

Challenge	Explanation	Remedy
High-resolution timing cost	Reading a high-resolution clock is itself an expensive operation — it should not be done in a tight loop.	Only time the operations that matter; aggregate counts rather than timing individual events.
Single-machine timing	Two timestamps on the same machine are directly comparable only if NTP is not running (NTP can step the clock backwards or forwards).	Use a monotonic clock for measuring durations — it never goes backwards regardless of wall-clock adjustments.
General-purpose OS scheduler	The OS scheduler can preempt a process at any point, making high-precision timing of short intervals unreliable.	Use many repetitions and statistical analysis to characterise the distribution — single measurements are not reliable.
Cross-machine timestamps	Clocks on different machines are unlikely to be synchronised and will drift at different rates.	Either synchronise clocks (NTP, PTP) and quantify the residual uncertainty , or design the experiment to avoid cross-machine timestamp comparisons.

7. Data Analysis & Presentation

Analysis process

After an experiment you have a set of log files — typically one per component or machine. Processing steps:

- **Parse** log files into a structured format
- **Identify events** of interest (e.g. request sent, response received)
- **Extract metrics** from relevant time windows (e.g. latency = response time – request time)
- **Aggregate** across repetitions to build distributions
- **Visualise** to check sanity and support conclusions

Plot rules

Every plot must be self-contained:

- Context (or reference) must be obvious from the plot alone
- Every axis labelled with **metric name and unit** (e.g. Latency/[ms])
- Multiple datasets → use a **legend**
- Axes start at zero **except**: absolute time axis, logarithmic axis, or when justified explicitly
- Use a **systematic colour mapping**: same implementation always gets the same colour across all plots

Visualisation types

Plot type	Python (seaborn)	When to use
Scatter plot (timeseries)	<code>sns.scatterplot(x='day', y='birth')</code>	Show raw data, outliers, trends
Histogram	<code>sns.histplot(x='birth', stat='density')</code>	Show the shape of the distribution, skew, and mode
KDE (Kernel Density Estimate)	<code>sns.kdeplot(x='birth')</code>	Smoothed version of a histogram, shows the shape of the distribution
CDF (Cumulative Distribution Function)	<code>sns.kdeplot(x='birth', cumulative=True)</code>	Show what proportion of the data is below a certain value. Easy to read
Violin plot	<code>sns.violinplot(x='weekday', y='birth')</code>	Show full distribution of a metric across categories. How a factor affects a metric
Heatmap / Contour map	(mentioned, not shown in code)	Show 2D relationships between variables. Useful for finding correlations

8. Python / Seaborn Code Patterns

All examples use pandas + seaborn + matplotlib. The dataset used in the lecture is daily births in Denmark (2019).

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv('data.csv', names=['day', 'value', 'factor'])

# 1. Timeseries scatter
sns.scatterplot(x=df['day'], y=df['value'])
plt.xlabel('Time/[days]'); plt.ylabel('Value/[unit]')

# 2. Histogram of distribution
sns.histplot(df, x=df['value'], stat='density', kde=False)

# 3. Smoothed KDE
sns.kdeplot(x=df['value'])

# 4. Cumulative distribution function
sns.kdeplot(x=df['value'], cumulative=True)

# 5. Factor-dependent violin plot
sns.violinplot(data=df, x='factor', y='value', color='lightsteelblue')

plt.savefig('result.pdf') # always save to PDF for quality
```

Always save plots as PDF or SVG rather than PNG — vector formats scale without loss and look professional in reports. Always label axes with both the metric name and its unit separated by a slash: Latency/[ms], Throughput/[req/s].

Lecture 11 — Budget Management

Software Technology for Internet of Things | SDU MSc Software Engineering | Exam notes

1. What is a Budget?

Definition: A budget represents the availability of a scarce resource.

- A budget can be **consumed** (spent down)
- Some budgets are **replenished** at a more or less consistent rate (e.g. solar energy per day, packet quota per day)
- Budgets can be **defining** for device operation — if the budget is exhausted the device cannot perform its function

IoT devices face **four primary budgets** that interact with each other through tradeoffs.

2. The Four Budgets

Budget	Restrictions	What it affects
Radio	Number of packets per day; rate of transmission (duty cycle); maximum packet size	How often and how much data can be sent. Consumers: data collection, actuation requests, configuration distribution, beacons, routing table maintenance, retransmissions.
Memory	RAM size; ROM size; Flash size; number of write cycles per cell; erase unit size	What data and code fits on the device; how to store data locally; the tradeoff between on-device processing (needs RAM) and raw data forwarding (needs radio).
Processing	Number of cores; availability of DMA and hardware crypto; clock frequency; IPC; integer vs single-precision vs double-precision floating-point capability	What computations can run independently; how fast data can be processed; the tradeoff between processing data locally vs transmitting it raw.
Energy	Battery capacity (lifetime); solar panel yield (per day); PSU maximum power draw	How long the device can operate; which components can be on simultaneously; dominated by radio then CPU then sensors.

Power consumption (low → high):

CPU: off < sleeping < idling < processing

Sensors: off < * on (* order depends on sensor type)

Radio: off < sleep < receive ≈ transmit

Relative magnitude: sensors << CPU << radio

Rule of thumb: the radio is the dominant energy consumer.

Minimising radio on-time is the highest-leverage optimisation.

3. Budget Tradeoffs & Interactions

Tradeoff	Description	Example
Processing vs Radio	Doing more computation on-device (more processing budget) reduces the amount of data that needs to be transmitted (less radio budget). Transmitting raw data saves processing but consumes radio.	Compute a running average on-device and only send one value per minute instead of 60 raw samples.

Processing vs Memory	More sophisticated algorithms need more RAM. If RAM is scarce, simpler algorithms must be used or data must be streamed rather than buffered.	A sliding-window average needs to store N values in RAM; a cumulative average needs only 2 values.
Radio vs Energy	Radio transmit/receive is typically the largest energy consumer. Reducing transmissions directly extends battery life.	Batch 8 samples into one packet: 8× fewer transmissions, dramatically lower energy consumption.
Latency vs Energy	Sleeping and waking in bursts saves energy but increases end-to-end latency. Staying awake gives low latency but drains the battery faster.	A smoke alarm must wake instantly (low latency); a soil moisture sensor can sleep for hours (low energy).
Data resolution vs Radio	Higher sampling rate and bit depth produce more data requiring more radio bandwidth. Reducing resolution or sample rate conserves radio budget.	Downsample from 100 Hz to 1 Hz for slowly-changing temperature.

4. Resolving Tradeoffs — The Approach

The situation: many budgets, many interdependent tradeoffs, magnitudes hard to reason about, no universal silver bullet.

Recommended process:

- **Step 1:** First and foremost, eliminate choices with significant downsides. Many tradeoffs resolve themselves by ruling out clearly bad options.
- **Step 2:** For the remaining tradeoffs that are genuinely ambiguous, experiment your way to a resolution — measure, don't guess.

This is why performance evaluation (Lecture 10) is directly relevant to budget management: budgets must be measured, not assumed.

5. Data Throttling Techniques

Data throttling reduces the number of transmissions — the primary lever for both **radio** and **energy** budgets.

Technique 1: Aggregate Values Instead of Raw Values

Instead of transmitting raw sensor readings, transmit a **running cumulative sum** (or other aggregate).

Raw transmission: t1: 10 t2: 12 t3: 11 t4: DROP ← budget blown t5: 14 Gap at t4: cannot safely reconstruct signal.	Cumulative aggregate: t1: 10 t2: 22 (10+12) t3: 33 (22+11) t4: DROP ← budget blown t5: 57 (33+11+13) Gap at t4: signal degrades gracefully in resolution.
--	---

Property	Raw transmission	Aggregate transmission
Values grow indefinitely?	No — each is independent	Yes — the cumulative sum grows
Number of values transmitted	One per sample	One per sample (same rate)
Can derive original signal?	Yes — directly	Yes — take differences between consecutive values
Effect of a dropped packet	Gap: cannot reconstruct that sample safely	Graceful degradation — lose temporal resolution for that period, but the surrounding values remain valid

Technique 2: Adaptive Sampling

Periodic sampling produces bandwidth proportional to frequency, regardless of how much the signal is changing. But not all samples are equally informative.

Adaptive sampling: only emit a value when it differs significantly from the last emitted value.

- Stable signal → few transmissions → radio/energy budget saved
- Rapidly changing signal → frequent transmissions → full fidelity preserved
- Timing-critical applications may still need periodic sampling — adaptive sampling trades temporal precision for budget

Technique 3: Batch Transmission

The sample-transmit-sleep pattern sends one packet per sample, most of which are nearly empty. Batching fills each packet before sending:

Sample-Transmit-Sleep:

[s1]→TX [s2]→TX [s3]→TX
[s4]→TX [s5]→TX ...

- ✓ Low latency
- ✗ Mostly-empty packets
- ✗ High per-sample TX overhead

Batch Transmission:

[s1][s2][s3][s4]→TX
[s5][s6][s7][s8]→TX ...

- ✓ Full packets – less overhead
- ✓ Better energy/radio efficiency
- ✗ Latency = $n \times \text{sample period}$

Latency penalty: with a batch size of n and sample period T , the worst-case latency from sensing to transmission is $n \times T$. Choose batch size based on the application's latency tolerance.

6. Power Throttling Techniques

Technique 4: Sleep Modes

Microcontrollers and peripherals have multiple power states with different wake-up times and power draws. Key insight: different subsystems have different power state requirements.

Power state	CPU	Peripherals	Typical use
Full active	Running	All on	Active computation, radio TX/RX
Idle	Halted (waiting for interrupt)	Partially on	Waiting for timer or interrupt
Sleep / Light sleep	Off	Some on (e.g. timer, ADC in DMA mode)	Periodic sampling without CPU spin
Deep sleep	Off	Minimal (wake timer only)	Long idle periods — maximum energy saving

Good practice: track the power requirements of all *currently active* code paths and continuously transition to the deepest sleep mode that satisfies all of them. TinyOS had abstractions to do this automatically.

ADC in DMA (Direct Memory Access) mode can sample a sensor and write results directly to memory without waking the CPU — a powerful technique for sensor-heavy applications.

Technique 5: Low-Power Listening (LPL)

Receiving radio signals consumes significant energy. LPL drastically reduces receive-side energy by duty-cycling the radio:

Receiver (battery-powered):
 zzzZZZ[CCA]zzzZZZ[CCA]zzzZZZ[CCA!]■■■AWAKE■■■RECEIVE■■■zzzZZZ
 ↑ activity detected

Transmitter (mains-powered):
 ■■■■■■■■■■[PREAMBLE.....]■■■DATA■■■
 ↑ preamble length = receiver sleep period

Key: CCA = Clear Channel Assessment
 Preamble ensures receiver wakes during transmission window.

Role	Behaviour	Energy profile
Receiver (battery)	Sleeps most of the time. Wakes periodically to perform a Clear Channel Assessment (CCA). If channel active: stay awake and receive. If clear: sleep again.	Radio on only for brief CCA windows → very low average receive power
Transmitter (mains/ample energy)	Before sending data, transmits a long preamble lasting at least one full sleep period of the receiver. The receiver will wake during this preamble, detect activity, and stay awake for the data packet.	Higher transmit energy, but this cost is borne by the energy-rich side

LPL is the right choice when: the receiver is on scarce energy AND transmissions are infrequent OR the transmitter has plentiful energy. The key tradeoff: transmit energy increases (longer preamble) to save receive energy. This is explicitly asymmetric — move the cost to whoever can afford it.

7. Budget Management Algorithm

When a budget is limited (e.g. LoRaWAN: 140 uplinks/day), you must distribute spending across time to avoid exhausting it early and going dark for the rest of the period:

```

1. Keep track of remaining_budget (e.g. packets left today)
2. Keep track of remaining_time (e.g. seconds until midnight)
3. Split remaining_time into chunks (e.g. 1 chunk = 1 hour)
4. Allocate budget to current chunk:
   chunk_budget = remaining_budget / chunks_remaining
5. For each event: decide if worth processing:
   if event.cost <= chunk_budget_remaining:
     process(event)
     chunk_budget_remaining -= event.cost
   else:
     defer_or_drop(event)
  
```

Step	Explanation
Track remaining budget	Count how many units of the scarce resource are still available (e.g. packets remaining today, mAh remaining in battery).
Track remaining time	Know how long until the budget resets or the device must last (e.g. seconds until midnight, days until battery replacement).
Split into chunks	Divide remaining time into equal intervals (e.g. one chunk per hour). This creates a natural pacing rhythm.
Allocate budget to current chunk	$\text{chunk_budget} = \text{remaining_budget} \div \text{chunks_remaining}$. This ensures even distribution even if past chunks were cheaper or costlier.
Gate events against chunk budget	Before processing an event (e.g. transmitting a packet), check if its cost fits in the remaining chunk budget. If not, defer or drop it.

This algorithm is adaptive: if earlier chunks underspent, later chunks get more budget; if they overspent, later chunks get less. The recalculation at each chunk boundary prevents early exhaustion without permanently penalising a brief burst of activity.

Lecture 12 — Annotating Identity, Time & Space

Software Technology for Internet of Things | SDU MSc Software Engineering | Exam notes

1. Naming

Naming is one of the most important yet simplest concepts in computer science. It is **not** about assigning a label to a concept — it is about being able to **reference** something.

A name has meaning only in combination with a **context**. The name alone does not identify the data; the context determines what the name resolves to.

name + context → data (reference)

Example 1: ['/home/asjo'] + ['.bashrc'] → /home/asjo/.bashrc

Example 2: ['myObject'] + ['print()'] → myObject.print()

Key insight: naming is about referencing, not labelling.

The name has no meaning without its context.

In IoT, naming is the foundation of identity management: you cannot give a device's data streams meaning until you can reliably reference the device that produced them.

2. Identity Management

A device **identity** is a name that allows us to:

- Associate a device **to** something — device 12 produces data in stream 17
- Associate something **to** a device — room U181 contains device 12

A device may have **multiple identities** for different purposes and exposure levels (e.g. a network address, a physical serial number, a user-facing nickname).

Source of identity	How obtained	Properties
1. Unique device number	Serial from a dedicated ID chip, the MCU, or a radio chip	Globally unique, hardware-burned, immutable. No human input required. Cannot be changed if compromised.
2. User-provided	Entered via mobile app or web interface during installation	Human-meaningful. Flexible. Requires a commissioning step. Can be changed if needed.
3. Physical location-derived	A GPS coordinate set captured at installation	Intrinsically meaningful for spatial queries. Becomes wrong if device is moved without re-provisioning.
4. Network-derived (logical location)	Through DNS, DHCP, or topology enumeration	Reflects the device's position in the network. Changes if topology changes. Not portable across networks.
5. Context-derived	Combination of what it is, where it is, and what events surround it	Richest form — can identify a device even without pre-provisioning. Requires substantial contextual knowledge to compute.

3. Point of Identification — Where is Identity Rooted?

Location	Description	Consequences
----------	-------------	--------------

Node (self-description)	The device itself stores and asserts its own identity. It is 'self-aware'.	Device can identify itself without external lookup. Identity survives network outages. Risk: if the device is compromised, its identity can be spoofed. Useful: a device can announce itself to any gateway on first contact.
Cloud	Identity is stored centrally. The cloud is the authoritative source of truth about what each device is.	Central control and revocation. Requires network access to resolve identity. Single point of failure for identity. All systems that need to identify the device must reach the cloud.
Private (user-held)	Identity data is stored with the end user, not the device or the vendor.	Privacy-preserving — vendor cannot track device usage. Hard to recover if user loses the data. Fits GDPR 'data minimisation' principle. Requires user to participate in any identity resolution.

Key questions to ask when choosing: Who needs to know the identity? Who can get the identity, and at what cost (bandwidth, latency, privacy)? What happens if the identity store is unavailable?

4. Timekeeping on Devices

A running device counts clock ticks — this counter converts to an uptime (time since boot). Raw timestamps in radio frames are derived from this counter on the originating device. By propagating timestamps across the network, devices can build a shared sense of time — enough for ordering events, even without perfect wall-clock synchronisation. For a deeper treatment: Vector clocks (logical ordering without wall-clock sync) and NTP (wall-clock synchronisation).
--

5. Point of Timestamping

A timestamp can be applied at six different points in the device → gateway → cloud pipeline. The choice has significant accuracy and tradeoff implications:

1	2	3	4	5	6
On device (at sampling)	On device (at transmit)	On gateway (at receive)	On gateway (at cloud push)	In cloud (at receive)	From cloud (at delivery)

Point	Stamped by	Accuracy	Key tradeoff
1. At sampling	Device	Most accurate — closest to the physical event	Requires the device to have a synchronised clock. Device clock drift introduces cumulative error.
2. At transmit	Device	Slightly less accurate — adds queuing/processing delay	Easier to implement than sampling-time stamps; avoids the need for an ADC-integrated clock.
3. At gateway receive	Gateway	Adds network propagation delay and queuing jitter	No device clock required. Cheap to implement. Not representative of when the physical event occurred.
4. At gateway cloud push	Gateway	Adds local processing delay on top of receive	Worst accuracy of the gateway options; simplest to implement (single timestamp per batch).
5. At cloud receive	Cloud	Adds full end-to-end latency (network + gateway processing)	Completely decoupled from physical event time. Sufficient only for coarse ordering or arrival-time tracking.
6. At cloud delivery	Cloud	Furthest from the physical event	Only useful for measuring system processing latency itself, not for event timing.

The earlier the timestamp is applied, the closer it is to the physical event — but the more it depends on the device having a reliable, synchronised clock. The best choice depends on the application's latency requirements and whether device clock synchronisation is feasible.

6. Sources of Time on a Device

Source	Description	Notes
1. Offset (relative time)	Just count ticks since boot — uptime only, no wall-clock time	Always available. Sufficient for ordering events on one device. Not comparable across devices without additional synchronisation.
2. GNSS (GPS / GLONASS / Galileo / BeiDou)	Atomic-clock accuracy from satellite signals	Very accurate (~100ns). Requires a GNSS receiver (cost, power). Multiple systems exist because of 'Selective Availability' — the US degraded GPS accuracy during the 1999 Kargil War, motivating independent systems (EU: Galileo, Russia: GLONASS, China: BeiDou).
3. USB cable at startup	Host computer pushes current time to device when connected	Simple, no extra hardware. Only works at boot/connect time. Drifts freely after disconnect.
4. Downstream push	Cloud or gateway pushes current time to the device	Requires network connectivity. Depends on the cloud's accuracy. Practical for devices that connect periodically.
5. In-channel broadcast	Time is included in the same radio channel as data traffic	No extra communication required. Accuracy limited by broadcast interval and propagation delay.
6. Backchannel broadcast	Time is broadcast on a dedicated separate radio channel	Can be more accurate and frequent than in-channel. Requires receiver support for the additional channel.
7. NTP (Network Time Protocol)	Standard internet time synchronisation protocol	~1–50ms accuracy over internet; sub-ms on LAN. Widely available. Requires IP network access.

Why so many GNSS systems? Selective Availability: the US historically degraded civilian GPS accuracy (up to ~100m) for military reasons (e.g. Kargil War 1999). This motivated other nations to build independent satellite navigation systems not subject to US control.

7. Spatial Annotation — Position

Spatial annotation associates a device with a location, enabling queries like 'show me all sensors in building X' and visualisations like floor-map heatmaps.

Triangulation

Given distances to enough known positions and sufficiently small measurement errors, the device's position can be determined. With exact distances the intersection is trivial; with measured distances it becomes an error-minimisation problem:

3 known positions (A, B, C) + 3 measured distances (dA, dB, dC):

A●■■■■dA■■■■■

■■■→ position (minimise distance errors)

B●■■■■dB■■■■■

■

C●■■■■dC■■■■■

With exact distances: trivial intersection of circles.

With measured distances: least-squares / optimisation problem.

Can run on-device or in the cloud.

Source of position	Type	How it works	Notes
Hardcoded in firmware/flash	Direct	A known coordinate is written to the device at manufacture or install time	Zero runtime cost. Becomes wrong if device is moved. Suitable for static infrastructure.
GNSS (GPS/GLONASS/Galileo/BeiDou)	Direct	Satellite ranging gives absolute coordinates	Accurate outdoors. High power cost. Poor indoors (signal blocked by roofs/walls).
Downstream push (manual)	Direct	A human enters or selects the device location via an app or web UI	Simple, flexible. Requires human intervention. Can be updated without new firmware.
USB at startup (manual)	Direct	Location pushed from connected PC at boot	Only available at provisioning time.
Signal strength from known neighbours	Indirect (triangulation)	Measure RSSI from neighbouring devices at known positions; use triangulation to estimate location	No GPS needed. Accuracy depends on node density and signal propagation model. Works indoors. Errors from multipath and absorption.

EXAM GUIDE — SDU IoT June 9, 2026

Exam date

**June 9,
2026**

Re-exam

**August
18,
2026**

MCQ part

**60 min
utes**

Written
part

**60 min
utes**

MCQ part (60 min) — NO AIDS

No aids of any kind allowed. Generative AI counts as an aid.

Question formats:

- **Multiple choice** — pick one correct answer
- **Multiple answer** — pick a subset of correct answers
- **Short text** — write up to a few lines

Typical question types:

- Definitions (e.g. 'What is aliasing?')
- Categorical answers (e.g. 'Which OSI layer handles routing?')
- Brief concept explanations

Administered via **digitaleksamen.sdu.dk**

Written part (60 min) — RESTRICTED AIDS

Typical question types:

- Explain an intricacy of a topic to demonstrate deep understanding
- Explain how code from your exercises worked
- Detail a tradeoff and its implications
- Explain how a concept works
- **Come up with a design and argue for your choices**

Hint from slides: *"Consider how you can demonstrate an understanding of the question subject."*

Upload as PDF via **digitaleksamen.sdu.dk**

Allowed aids (written part only)

- Any course material in **PDF form** from itslearning
- Scientific articles linked from itslearning (PDF)
- Course page / Discord content converted to **PDF**
- Student-driven presentation in PowerPoint and/or PDF
- Your own C code from exercises and the **esp-idf** git repository
- Your own **handwritten or typed notes** (as PDF)
- A PDF viewer, browser for the test page, and PDF creation tool

NOT allowed

- **Generative AI** (e.g. ChatGPT, Claude, Copilot)
- Any browser tabs not pointing to the test page
- Any programs other than the three listed above

Quick-Reference: Key Definitions for the Exam

Term	One-line definition
IoT (course def.)	A cyber-physical system of individually and globally addressable devices with significant logic running elsewhere
Budget	Availability of a scarce resource that can be consumed and sometimes replenished
COP Theorem	Under overload you can guarantee Completeness OR Timeliness, not both
Nyquist theorem	Must sample at $\geq 2\times$ the highest signal frequency to reconstruct without aliasing
Aliasing	False lower-frequency artefact from sampling below the Nyquist rate
SNR	Signal-to-Noise Ratio — fraction of the signal that is not noise
Noise floor	Baseline noise level present even with no intentional signal
Transducer	Device that converts energy from one form to another (e.g. heat $\rightarrow$ voltage)
ADC	Analog-to-Digital Converter — samples a continuous voltage into a discrete integer
PWM duty cycle	Fraction of a period that a digital output is HIGH; controls average power
OSI Layer 1–4	L1: Physical (bit), L2: Data Link (frame), L3: Network (packet), L4: Transport (segment)
Frame	Unit of data at L2; includes header (source, dest, type, size), payload, CRC
CRC	Cyclic Redundancy Check — checksum appended to frames; detects corruption
CSMA/CA	Carrier-Sense Multiple Access / Collision Avoidance — listen before transmit
LPL	Low-Power Listening — receiver sleeps, wakes periodically to check channel
Pub/Sub	Pattern where producers publish to topics on a broker; consumers subscribe
MQTT QoS	0=at-most-once, 1=at-least-once, 2=exactly-once
Narrow waist	Shared information model interface between applications and buildings
RDF triple	Subject $\times$ Predicate $\times$ Object — atomic fact in an ontology store
Actor model	Concurrent unit with state, mailbox, and behaviour; communicates by message passing
GenServer call/cast	Call = synchronous (blocks for reply); Cast = asynchronous (fire and forget)
Supervision strategy	one_for_one / one_for_all / rest_for_one — how to restart on child crash

Let it crash	Elixir/BEAM philosophy: allow processes to crash; supervisors restart with fresh state
Adaptive sampling	Only emit a value when it differs significantly from the last emitted value
Batch transmission	Buffer n samples into one packet; trades latency for energy/radio efficiency
DIKW	Data → Information → Knowledge → Wisdom (apply meaning, context, experience)
Identity source	Unique number / user-provided / location-derived / network-derived / context-derived
Timestamp point	At sampling / at transmit / at gateway receive / at cloud receive — accuracy decreases
Triangulation	Determine position by measuring distances to known anchor points
Selective Availability	US policy of intentionally degrading GPS accuracy — motivation for Galileo/GLONASS
Performance eval cycle	Requirements → Analysis → Design → Implementation → Verification → PE (feedback loop)
Factor	A parameter deliberately varied across experiments to study its effect on a metric
Workload	Predefined sequence of inputs fed to a system at predetermined times
Experimental goal	First step of PE; defines what you are trying to learn; avoid bias
Black swan	An event models say is impossible because the supporting data was removed as an outlier
Cloud/Fog/Edge/Mist	Processing tiers: Cloud=high power/latency; Mist=on-device, low latency, resource-scarce

Exercise Notes — SDU IoT

Software Technology for Internet of Things | SDU MSc Software Engineering | All 9 exercise sets summarised for exam

Level	Exercise title	Key tech	Total XP
L2	Drinking the Elixir	Elixir, LiveBook, GenServer, Actor model	70+
L3	Metadata: Semantic Madness	RDF/SPARQL, Neo4j/Cypher, SHACL, Brick schema	55+
L4	To C or Not to C	C, GCC, arrays, pointers, structs, TSP	60+
L5	Toolchain: First Steps	ESP-IDF, ESP32, GPIO, UART, serial comms	80+
L6	Sampling Physical Phenomena	ADC, thermistor, FreeRTOS tasks, queues	65+
L7	Radio: Wireless Magic	WiFi, TCP sockets, MQTT, ESP-IDF networking	35+
L8	Stream Processing	MQTT, Elixir streams, moving averages, Flows	75+
L9	Performance Evaluation	Test harness, BEAM, latency evaluation, plots	50+
L9b	Budget Management	FreeRTOS tasks, queues, running avg, thresholding	25+

Exercise Set — Drinking the Elixir (L2)

Quest 0: Install LiveBook

5XP

Install LiveBook (depends on Elixir → depends on Erlang). Access at <http://localhost:4000/>. Enable **Render Ligatures** in settings.

Quest 1: Kickstart

10XP

Work through the 10-module getting-started notebook from the elixir-intro repository.

Quest 2: Getting Acquainted

15XP

Step through the livebook-demos workbooks:

- Basic data types
- Control structures
- Pattern matching
- Pipelines
- Primitive concurrency

Quest 3: Understanding the LiveMD Format

5XP

LiveMD files are valid **Markdown**. Open a .livemd file in a text editor to see the structure — sections, code cells, and Elixir blocks are embedded in standard Markdown.

Quest 4: Sources of Information

5XP

The documentation.livemd file links to: live-coding videos (Elixir School), module docs (hexdocs.pm), and the official language guide (elixir-lang.org).

Quest 5: Experimenting with Pipelines

10XP

Key insight from the pipeline workbook:

- Removing Enum.to_list and adding it back produces a list instead of a MapSet
- Disabling String.downcase() causes case-sensitive matching — different results

The |> pipe operator passes the left expression as the first argument of the right function. Pipelines make data transformation readable top-to-bottom.

Quest 6: Working with Functions

5XP

plotting-functions.livemd: Comprehensions generate lists; VegaLite.mark(:area) fills below a curve. The dynamic plot: interval = update frequency; 2nd arg to listen = event name; fun returns a list of data points; push adds to the live chart; window = max points shown.

Quest 7: Servers Everywhere!

10XP

generic-servers.livemd — key questions:

- 'Callback' = function called by the framework when an event occurs; the name comes from the inversion of control
- call vs cast: call = synchronous (need a response); cast = async (fire and forget)
- Only calls → server becomes a bottleneck; every caller blocks waiting for a response
- Registry supports: unique keys, duplicate keys, and partition modes

Quest 8: World of Zuul

20XP

World of Zuul is a text adventure implemented as a multi-player system using GenServers. The 'say' command is broken — players can only hear themselves.

Fix approach:

- Each Space (room) GenServer needs to track which players are currently in it
- Update player → space mapping on every 'go' command
- When a player says something, look up all players in the same space and append the message to each of their output frames using their client ids

Key concept: the origin field in the event stream is the client id — unique to the browser session. This enables the multiplayer architecture.

Exercise Set — Metadata: Adventures in Semantic Madness (L3)

Quest 1: Exploring the Lecture Example

5XP

Familiarise with the building model in the rdf.livemd or neo4j.livemd files. Understand the high-level structure before moving to Quest 2.

Quest 2: Exploring the Source

5XP

Work through both RDF and Neo4j data models. The same exercises apply to both — compare how each model represents the same information.

Strategy choice: tackle one data model fully first, then the other (depth-first), OR do the same exercise in both models before moving on (breadth-first). The breadth-first approach reveals the comparison most clearly.

Quest 3: A New Query

5XP

Write a query that lists all rooms that: (i) are contained within a building, and (ii) have a live data stream reporting in Kelvin.

SPARQL:

```
SELECT ?room WHERE {
  ?building a brick:Building .
  ?building brick:hasPart ?room .
  ?room a brick:Room .
  ?stream brick:pointOf ?room .
  ?stream brick:hasUnit unit:K .
}
```

Cypher:

```
MATCH (b:Building)-[:hasPart]
->(r:Room)-[:pointOf]
-(s:Stream {unit: 'K'})
RETURN r.name
```

Quest 4: Universe Expansion

10XP

Add a Building type, create an instance, connect it to floors, and write a query to verify. Key steps:

- Define brick:Building as a class (OWL: owl:Class, RDFS: rdfs:Class, Neo4j: label)
- Add brick:hasPart or brick:contains relationship from Building to Floor
- Instantiate: demo:MyBuilding a brick:Building; demo:MyBuilding brick:contains demo:floor1
- Verify with SELECT ?floor WHERE { demo:MyBuilding brick:contains ?floor }

Quest 5: Temperature Actuator

15XP

Add thermostat support — each room needs: temperature sensor UUID, setpoint UUID, radiator valve (actuator) UUID.

Key design decisions:

- Add brick:TemperatureActuator and brick:TemperatureSetpoint to vocabulary
- Use brick:pointOf to link actuators/setpoints to rooms (same as sensors)
- Consider whether brick:provides is correct or brick:pointOf should be used for all
- Query extracts all three UUIDs per room so an application can implement a thermostat
- A room missing one of the three produces no row in the result (inner join semantics)

Quest 6: Schema

20XP

Problem: no validation exists — any triple can be added. A model checker would:

- Check all subjects/objects of brick:pointOf are Equipment/Location respectively
- Verify every Temperature_Sensor has exactly one brick:unit
- Ensure no Room lacks the required thermostat components

Missing rules: cardinality constraints, domain/range enforcement, required component presence.

- **RDF**: implement using SHACL shapes (sh:NodeShape, sh:property, sh:minCount)
- **Neo4j**: implement as Cypher queries that find violating nodes (negative queries)

You do not need to implement all rules — just enough to cover the key relationships before the steps become repetitive.

Exercise Set — To C or Not to C (L4)

Quest 1: Making that First Compile

5XP

Compile, execute, and observe Hello World from a terminal:

```
gcc hello.c -o hello
./hello
# Expected: Hello, World
```

Quest 2: Offsetting the Index

10XP

Demonstrates pointer arithmetic to create a 1-indexed array from a 0-indexed one.

```
int a[SIZE]; // 0-indexed: a[0]..a[SIZE-1]
int* b = &a[0]; // b points to a[0]
b--; // b now points to a[-1] logically
// b[1] == a[0], b[SIZE] == a[SIZE-1]
// b[0] reads garbage (one before a) – undefined behaviour
```

Explanation of step 5 output: Uninitialized stack memory — contains whatever was in that stack frame previously. No guarantee of zeroes.

b[0] in step 16: reads one int before a[0] — garbage value, undefined behaviour.

Quest 3: In Support of a Hero (TSP)

30XP

Implement the Travelling Salesman Problem (TSP) for 15 Danish cities. The data structures are given — you implement the search algorithm.

Key insight: TSP is NP-hard — exact solution requires trying all permutations ($15! \approx 1.3$ trillion). For this exercise, a greedy nearest-neighbour heuristic is acceptable:

- Start at city 0
- At each step: go to the nearest unvisited city
- Return to start after all cities visited
- Print the route and total distance

For a better solution: implement 2-opt swaps to improve the greedy route.

Quest 4: The Final Test — Code Analysis

15XP

Analyse the mystery code that casts an `int[]` to a `long[]`:

```
int* values = malloc(12 * sizeof(int));
for (int i=0; i<12; i++) values[i] = i;
// values = [0,1,2,3,4,5,6,7,8,9,10,11]
// Each int = 4 bytes. Total = 48 bytes.

// Reinterpret as long* (8 bytes each on 64-bit):
// ((long*)values)[0] = bytes of values[0] + values[1]
// = 0x00000000 00000001 (little-endian)
// = 4294967296 + 0 = 4294967296 ... (depends on endianness)
// Prints 12/2 = 6 pairs of adjacent ints interpreted as one long
```

On a little-endian 64-bit system: $((\text{long}^*)\text{values})[i]$ combines two consecutive ints. $\text{values}[0]=0$ and $\text{values}[1]=1 \rightarrow 0x00000000100000000 = 4294967296$. This exercise demonstrates: pointer aliasing, type punning, platform dependency, and that $\text{sizeof}(\text{long}) = 2 \times \text{sizeof}(\text{int})$ on 64-bit systems.

Exercise Set — Toolchain: A Device Takes Its First Step (L5)

Quest	Title	XP	Key outcome
0	Equipment Capture	0	Get your ESP32 device
1	Early Steps	15	Install esp-idf v5.1.2, build/flash/monitor hello_world
2	Blinking Away	10	GPIO output, TaskBlink with vTaskDelay, two LEDs at different rates
3	Serial Count	15	UART full API, read integer from serial, count down
4	Reporting Changes	5+20	GPIO input polling for rise/fall; bonus: interrupts
5	Command and Control	10	Dispatch table, function pointers, command parser, pin-high/low/read
6	Bridging Devices	10+20	Two ESP32s communicating via GPIO; bonus: laptop-side code

Key code patterns from these exercises

FreeRTOS task creation:

```
xTaskCreate(
    TaskBlink, // function
    "Blink", // name
    4096, // stack size
    NULL, // pvParameters
    1, // priority
    NULL // task handle
);
```

GPIO setup:

```
gpio_reset_pin(PIN);
// Output:
gpio_set_direction(PIN,
    GPIO_MODE_OUTPUT);
gpio_set_level(PIN, 1);
// Input:
gpio_set_direction(PIN,
    GPIO_MODE_INPUT);
int v = gpio_get_level(PIN);
```

Command dispatch table:

```
typedef void(*command_t)
(int, char*[]);

typedef struct {
    char* name;
    command_t fn;
} dispatch_entry_t;

#define COMMAND_COUNT 1
dispatch_entry_t table[] = {
    {"echo", command_echo},
};
```

UART / serial delay:

```
#define DELAY \
(1000/portTICK_PERIOD_MS)
vTaskDelay(DELAY); // 1 sec
```

Quest 5's dispatch table pattern is directly relevant to the exam — it combines function pointers, structs, and arrays from the C lecture into a practical embedded design pattern.

Exercise Set — Sampling Physical Phenomena (L6)

Quest	Title	XP	Key outcome
1	Seeing Potential	15	ADC oneshot API, potentiometer, sample() function
2	Guessing Game	10+1	Game loop, ADC range mapping, binary search UX
3	Getting Real	10	LMT86 thermistor, voltage→temperature conversion, convert()
4	Cracks are Forming	15	Visualise sensor data: scatter plot, heatmap, live Elixir or Python
5	Root-Causing Noise	5	Wall power vs battery: different noise floors due to PSU ripple
6	Laying Pipes	10	FreeRTOS queue between two tasks: TaskSample → TaskTransmit

Key concepts & code patterns

Temperature conversion (LMT86):

```
// raw → voltage (3.3V ref, 12-bit ADC):
float v = raw * 3.3f / 4096.0f;

// voltage → °C (LMT86 datasheet §8.3.1):
// T = (V - 2.1422) / (-0.01386)
// (approx linearisation)
int8_t t = (int8_t)
((v - 2.1422f) / (-0.01386f));
```

FreeRTOS queue between tasks:

```
QueueHandle_t q =
xQueueCreate(10, sizeof(int));

// Sender:
xQueueSendToBack(q, &val, 10);

// Receiver:
int v;
xQueueReceive(q, &v, 10);
```

Noise sources (Quest 5):

- Wall power: PSU switching noise (50/60 Hz ripple) couples into analog signal → higher noise floor
- Battery: no PSU ripple → lower, cleaner noise floor

Pipeline architecture (Quest 6):

```
TaskSample
samples ADC
xQueueSendToBack(tx_queue)
```

```
TaskTransmit
xQueueReceive(tx_queue)
printf over UART
```

ADC type: use **int** for raw ADC value (always non-negative; fits in 12 bits on ESP32).

Exercise Set — Radio: That Old Wireless Magic (L7)

Quest	Title	XP	Key outcome
1	Establish an Access Point	5	Phone/laptop hotspot, SSID + password
2	Obtaining an IP Address	10	esp-idf WiFi station example, Kconfig, hl_wifi module
3	Uplink	15	TCP socket to laptop, hl_wifi_tcp_connect, hl_wifi_tcp_tx, counter
4	Temperature Uplink	5	Replace counter with thermistor readings over TCP

Key code patterns

WiFi module interface pattern:

```
// hl_wifi.h
typedef void(*connect_callback_t)(void);
typedef struct sockaddr_in
sockaddr_in_t;

void hl_wifi_init(
connect_callback_t* cb);
sockaddr_in_t hl_wifi_make_addr(
char* ip, uint16_t port);
int hl_wifi_tcp_connect(
sockaddr_in_t addr);
void hl_wifi_tcp_tx(
int sock, void* buf, uint16_t len);
```

TCP transmit loop (partial sends):

```
void hl_wifi_tcp_tx(
int sock, void* buf,
uint16_t len) {
uint16_t offset = 0;
while (offset < len) {
uint16_t rem = len - offset;
int sent = send(sock,
(char*)buf + offset,
rem, 0);
if (sent < 0) { /* error */ }
offset += sent;
}
```

send() may transmit fewer bytes than requested. Always loop until all bytes are sent.

Kconfig: The Kconfig.projbuild file defines menu entries that appear in 'idf.py menuconfig' and generate #defines in sdkconfig.h. This is how SSID/password are configured without hardcoding.

Exercise Set — Stream Processing: Pushing and Transforming Data (L8)

Quest	Title	XP	Key outcome
1	Install MQTT Broker	5	Mosquitto install and test; mosquitto_sub/pub
2	Signal Generator	10	Step through mqtt-signal-generator.livemd; understand concurrency model
3	Moving Averages	10	mqtt-signal-mavg.livemd; understand window, lost message consequences
4	Absolute Humidity	10	Compute Habs from T and Hrel, publish as new MQTT stream
5	Topic Structure & Latency	20	Design topic hierarchy; measure round-trip latency; discuss structure
6	Data Flows	10	flows.livemd; compare Flow vs MQTT/GenServer approach
7	Chatty Room (ESP32 MQTT)	15	ESP32 joins public MQTT broker; implements chat over TOPIC

Key concepts & answers

Quest 2 — Signal Generator concurrency: Each configuration spawns one BEAM process. n configurations → n+1 processes (n generators + 1 supervisor). Concurrency achieved through BEAM lightweight processes and MQTT topics as queues.

Quest 3 — Moving Average:

- Window = the last N values used to compute the average
- Values in the window are raw sensor readings (same unit as input)
- Lost message → window contains stale data for that slot → average is slightly off until the slot is overwritten

Quest 4 — Absolute Humidity formula:

```
# Habs = absolute humidity in g/m³
# T = temperature in °C
# Hrel = relative humidity in %
Habs = 6.112 * :math.exp(17.67*T / (T+243.5)) * Hrel * 2.1674 / (273.15+T)
```

Quest 5 — Topic Structure Design

Process	Publishes to	Subscribes to
Laptop uptime publisher	home/laptop/uptime	—
Room temp publisher (°C)	home/room1/temperature/celsius	—
Celsius→Fahrenheit converter	home/room1/temperature/fahrenheit	home/+/temperature/celsius
Absolute humidity	home/room1/humidity/absolute	home/+/temperature/celsius AND home/+/humidity/relative

Latency formula: end-to-end = publish → broker → subscribe → process → publish → broker → subscribe. Moving to a friend's laptop adds network hop latency × 2.

Quest 7 — ESP32 MQTT Chat (key snippets)

```
#define BROKER_URL "mqtt://broker.hivemq.com"
#define TOPIC "org/sdu/course/iot/year/2026/chat/channel/42"

// Connect:
esp_mqtt_client_config_t cfg = {.uri = BROKER_URL};
client = esp_mqtt_client_init(&cfg);
esp_mqtt_client_register_event(client, ESP_EVENT_ANY_ID, handler, NULL);
esp_mqtt_client_start(client);

// On MQTT_EVENT_CONNECTED: subscribe + create TaskChat
esp_mqtt_client_subscribe(client, TOPIC, 1);

// On MQTT_EVENT_DATA: print received message
printf("%.s\n", event->data_len, event->data);

// In TaskChat: publish formatted message
sprintf(buffer, "%s: %s\n", nick, line);
esp_mqtt_client_publish(client, TOPIC, buffer, 0, 1, 0);

// nick is char[10] → max 9 chars + null terminator
```

Exercise Set — Performance Evaluation: Streaming Humidity (L9)

Quest	Title	XP	Key outcome
1	Test Harness	10	record.livemd; understand how to instrument BEAM processes
2	Initial Setup	10	Reproduce L8 Signal Gen → Moving Avg → Abs Humidity pipeline
3	End-to-End Latency	30	Full PE methodology applied to MQTT pipeline

Quest 3 — Answering the Methodology Questions

Question	Answer
End-to-end latency	Time from a value being published by the Signal Generator to the corresponding result being received by the Client. Includes: broker routing, moving average processing, abs humidity calculation, second broker routing.
Result form	A distribution (not a single number) — histogram, KDE, CDF, or violin plot. Report mean, median, and tail percentiles (p90, p99).
System definition	Signal Generator(s) → MQTT Broker → Moving Average → MQTT Broker → Abs Humidity → MQTT Broker → Client. Stubs: the MQTT broker (external service).
Relevant services	Publish a sensor reading; receive a computed humidity value.

Metrics	End-to-end latency [ms]; throughput [messages/s].
Parameters	MQTT broker (local Mosquitto), window size, number of rooms, machine specs.
Factors	Number of rooms (e.g. {1, 2, 4, 8}); window size (e.g. {1, 5, 10, 20}).
Workload	Signal generator publishes n messages at a fixed rate (e.g. 1 msg/s per room). Run for 60s per configuration.
Experimental design	Full-factorial over factors; repeat each combination 5–30 times; instrument with timestamps at publish and receive.
Instrumentation	Timestamp each published message (T1) and each received message (T2). Latency = T2 – T1.
Post-processing	Parse log files; match messages by ID; compute T2–T1 for each; aggregate into a distribution.
Visualisation	CDF plot (easy to read percentiles); violin plot (compare across factor levels).
Conclusion	Expected: latency grows with number of rooms (more broker routing) and window size (more buffering); tail latency grows faster than median with load.

Exercise Set — Budget Management: Getting the Most for Your Money (L9b)

Quest	Title	XP	Key outcome
1	Running Average	10+5	Add TaskAvg stage between Sample and Transmit; stage_interface_t pattern
2	Thresholding	10	Add TaskThreshold for adaptive sampling; fix watchdog race condition

Complete pipeline architecture

Sample → [avg_queue] → RunningAverage → [thres_queue] → Threshold → [tx_queue] → Transmit

Queues: avg_queue (size AVG_BUFFER_SIZE=10)
thres_queue (size THRES_BUFFER_SIZE=10)
tx_queue (size TX_BUFFER_SIZE=10)

```
stage_interface_t avg_pair = {avg_queue, thres_queue};
stage_interface_t thres_pair = {thres_queue, tx_queue};
```

Key code patterns

stage_interface_t pattern:

```
typedef struct {
    QueueHandle_t input;
    QueueHandle_t output;
} stage_interface_t;

// In a task:
stage_interface_t* pair =
(stage_interface_t*)pvParameters;
QueueHandle_t in = pair->input;
QueueHandle_t out = pair->output;
```

Threshold / adaptive sampling:

```
#define THRESHOLD 100
int last = 1<<16; // > max ADC

// On new value from queue:
int diff = current - last;
if (diff < 0) diff = -diff;
if (diff > THRESHOLD) {
    xQueueSendToBack(out,
    &current, 10);
    last = current;
}
```

Running average (circular buffer):

```
#define AVG_WINDOW_SIZE 7
int buffer[AVG_WINDOW_SIZE] = {0};
uint8_t index = 0;
int sum = 0;

// On new value:
int old = buffer[index];
buffer[index] = new_val;
sum -= old;
sum += new_val;
index = (index+1) % AVG_WINDOW_SIZE;
int avg = sum / AVG_WINDOW_SIZE;
```

Watchdog fix (Quest 2, step 6):

- Symptom: application crashes with watchdog timeout
- Cause: tasks with priority 1 starve the idle task (which feeds the FreeRTOS watchdog)
- Fix: change priority from 1 to 0 for all xTaskCreate calls

Bonus (Quest 1, step 7) — why pre-fill buffer?

- Avoids the transient startup period where the average is computed over fewer than AVG_WINDOW_SIZE values
- The average is valid from the very first output

The running average + threshold pipeline directly maps to the budget management lecture concepts: running average reduces noise, threshold implements adaptive sampling (only transmit significant changes), and queues between tasks implement the stream processing node model.

Elixir Intro — Module Notes

github.com/aslakjohansen/elixir-intro | SDU IoT Course | All 10 modules summarised for exam

Two paths:

- **Beginner** (never programmed before): Module 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10
- **Advanced** (know at least one language): Module 8 → 9 → 10

Modules 1–7 cover Elixir fundamentals. Modules 8–10 are the practical exercises the IoT course focuses on. **All 10 modules are summarised here.**

Module 1: Values, Tuples and Pattern Matching

Beginner path

Values & expressions

All valid Elixir code evaluates to a value. Numbers, strings, and atoms are primitive values.

```
"Hello, World" # => string
42 # => integer
1 + 2 # => 3
:math.pi # => 3.14159...
:ok # => atom
```

Tuples

Fixed-length ordered sequences. A single value even when containing many.

```
{1, 2} # 2-tuple
{"a", 1} # mixed types
{1, {2, 3}} # nested
{:ok, result} # common pattern
{:error, :neg_r} # error pattern
```

Pattern matching (= operator)

The = operator is a *match*, not assignment. Left side is a pattern; right side is the value to match against.

```
# Bind variable:
pi = :math.pi

# Destructure tuple:
{:ok, area} = {:ok, 3.14}
# area = 3.14

# _ = wildcard (ignore):
{:ok, _, e} = {:ok, 1, 2}
# e = 2

# Match fails → MatchError:
{:ok, x} = {:error, 404}
```

Pattern matching is used everywhere in Elixir — function heads, case expressions, receive blocks, and variable binding all use it.

Module 2: Booleans, if, String and case

Beginner path

Booleans

```
true false
# Operators:
a and b # both true
a or b # at least one
not a # invert
# Comparisons → boolean:
x == y x != y
x > y x >= y
x < y x <= y
```

case

Pattern matches against multiple clauses; first match wins.

```
case result do
{:ok, value} ->
IO.puts(value)
{:error, msg} ->
IO.puts("Error: #{msg}")
_ ->
IO.puts("Unknown")
end
```

if / else

if is an *expression* — it returns a value.

```
{:ok, area} =
if r < 0 do
{:error, :negative_r}
else
{:ok, :math.pi * r*r}
end
```

String interpolation

```
r = 5
"radius is #{r}"
# => "radius is 5"

# String module:
String.upcase("hello") # HELLO
String.split("a b", " ") # ["a","b"]
String.downcase("HELLO") # hello
```

Atoms start with ':' — they are global constants. Equal only to themselves. Never garbage-collected. Common: `:ok`, `:error`, `:true`, `:false`, `:nil`

Module 3: Functions

Beginner path

Anonymous functions

```
# Define:
f = fn x -> 1.2*x*x - 1.3*x + 0.9 end

# Call with . operator:
f.(10) # => 109.9

# Pipe through (identity fn):
passthrough = fn x -> x end
passthrough.(42) # => 42
```

Named functions (module)

```
defmodule Circle do
def area(r) do
:math.pi * r * r
end

def circumference(r) do
2 * :math.pi * r
end
end

Circle.area(5) # => 78.5
Circle.circumference(3) # => 18.8
```

Capture operator &

Shorthand for anonymous functions.

```
# &1 = first argument
double = &(&1 * 2)
double.(5) # => 10

# Capture named function:
Enum.map(list, &Fibonacci.index/1)
# equivalent to:
Enum.map(list, fn n ->
Fibonacci.index(n) end)
```

Exercise outcome

Module 3 exercise: implement circumference function, generate dataset over a range of radii, plot using VegaLite.

```
circumference = fn r ->
2 * :math.pi * r
end

data = 1..20
|> Enum.map(fn r ->
%{"r" => r,
"c" => circumference.(r)}
end)
```

Module 4: Lists, Higher-Order Functions, Pipelining & Modules

Beginner path

Lists

```
l = [0, 1, 2, 3, 4]

# Head/tail destructure:
[h | t] = l
# h = 0, t = [1,2,3,4]

# Prepend:
[-1 | l] # => [-1,0,1,2,3,4]

# Index:
Enum.at(l, 3) # => 3
```

Higher-order functions

```
# map: transform each element
Enum.map([1,2,3], fn x -> x*2 end)
# => [2, 4, 6]

# filter: keep matching elements
Enum.filter([1,-2,3], fn x -> x>=0 end)
# => [1, 3]

# reduce: fold to single value
Enum.reduce([1,2,3], 0, fn x,acc ->
acc + x end)
# => 6
```

Pipelines (|>)

Passes result of left as first argument of right. Avoids nested function calls.

```
# Without pipe:
Enum.sum(Enum.filter(
Enum.map(l, fn x -> x+1 end),
fn x -> x >= 0 end))

# With pipe (same result):
l
|> Enum.map(fn x -> x + 1 end)
|> Enum.filter(fn x -> x >= 0 end)
|> Enum.sum()
```

Modules

Group related functions. Call with `Module.function(args)`.

```
defmodule MathUtils do
  def square(x), do: x * x
  def cube(x), do: x * x * x
end

MathUtils.square(4) # => 16
```

Enum.to_list/1 converts a *MapSet* or *range* to a *list*.
MapSet.new/1 creates a *set* (unique values only). *1..10* creates a *range*.

Module 5: Maps

Beginner path

Map basics

```
# Create:
m = %{"jan" => 31, "feb" => 28}

# Lookup:
Map.get(m, "jan") # => 31
Map.get(m, "xyz") # => nil
Map.get(m, "xyz", 0) # => 0 (default)

# Add / update:
m = Map.put(m, "mar", 31)

# Check key:
Map.has_key?(m, "jan") # => true
```

Atom key shorthand

```
# Atom keys use : shorthand:
c = %{r: 255, g: 0, b: 128}

# Access with . notation:
c.r # => 255

# Or Map.get:
Map.get(c, :r) # => 255
```

Tuple keys (multi-language)

```
m = %{
  {:en, "jan"} => 31,
  {:da, "jan"} => 31,
  {:da, "feb"} => 28,
}

Map.get(m, {:da, "jan"}) # => 31
```

Pattern matching with maps

```
# Match subset of keys:
%{x: x_val} = %{x: 3.14, y: 2.7}
# x_val = 3.14

# In function heads:
def greet(%{name: name}) do
  "Hello #{name}"
end
```

nil represents the absence of a value. *Map.get* returns *nil* for missing keys unless a default is provided.

Module 6: Colors (GenServer intro)

Beginner path

This module introduces **GenServer** through a practical example: a colour mixer driven by three sliders (R, G, B). The GenServer renders an SVG square updated in real time.

Key patterns introduced:

- **Interface functions** — called by external code to start or message the server
- **Callback functions** — called by the framework when a message arrives
- **handle_cast** — async update (no reply needed)
- **Kino.Frame.render** — update live UI component

[illegible]

Module 7: Structs Beginner path

Structs — typed maps

A struct is a map with a known, fixed set of keys. Defined inside a module with `defstruct`.

```
defmodule Person do
  defstruct [:name, :age,
            :nationality, :alive?]

  def first_name(person) do
    person.name
    |> String.split(" ")
    |> Enum.at(0)
  end
end

# Create an instance:
p = %Person{
  name: "Keira Knightley",
  age: 39,
  nationality: :english,
  alive?: true
}

Person.first_name(p) # => "Keira"
p.alive? # => true
```

Updating struct fields

```
# Map.put works on structs:
p = Map.put(p, :age, p.age + 1)

# Struct update syntax (cleaner):
p = %{p | age: p.age + 1}

# Can add arbitrary keys
# (becomes a plain map):
p = Map.put(p, :cpr, "1234")
# Note: __struct__ key is now
# visible - it marks the type
```

Exercise: birthday function

```
defmodule Person2 do
  defstruct [:name, :age,
            :nationality, :alive?]

  def first_name(person) do
    person.name
    |> String.split(" ")
    |> Enum.at(0)
  end

  # Birthday: increment age
  def birthday(person) do
    %{person | age: person.age + 1}
  end
end
```

String.split(s, " ") splits on spaces. Enum.at(list, 0) gets the first element.

Module 8: Circles (Structs + Higher-Order Functions)

Advanced path

Starting point for the **advanced path**. Combines structs, lists, higher-order functions, and pipelines on a set of circles with mixed-sign radii.

Circle struct

```
defmodule Circle do
  defstruct [:x, :y, :r]
end

circle = %Circle{
  x: 1, y: 2, r: 3}
circle.r # => 3
```

Exercise 2: total area

```
total_area = circles
|> Enum.map(fn c ->
  :math.pi * c.r * c.r
end)
|> Enum.sum()
```

Exercise 1: fix negative radii

```
circles = Enum.map(circles,
  fn c ->
    if c.r < 0 do
      %{c | r: -c.r}
    else
      c
    end
  end)

# or more concisely:
circles = Enum.map(circles,
  fn c -> %{c | r: abs(c.r)} end)
```

Exercise 3: double-area radius

Find r_{out} such that $area(r_{out}) = 2 \times area(r_{in})$:

```
# A(r) = pi * r^2
# A(r_out) = 2 * A(r_in)
# pi * r_out^2 = 2 * pi * r_in^2
# r_out = sqrt(2) * r_in

double_r = fn r ->
  :math.sqrt(2) * r
end

# Verify:
r_in = 3.0
r_out = double_r.(r_in)
# area(r_out) ≈ 2 × area(r_in) ✓
```

Module 9: Fibonacci Sequence (Concurrency: fork-join)

Advanced path

Fibonacci — recursive definition

```
defmodule Fibonacci do
  def index(0), do: 0
  def index(1), do: 1
  def index(n)
  when is_integer(n) and n > 1 do
    index(n-1) + index(n-2)
  end
end

0..39 |> Enum.map(&Fibonacci.index/1)
```

Sequential vs concurrent

Sequential: one at a time.

```
Enum.map(ns, &Fibonacci.index/1)
```

Concurrent (fork-join): spawn one process per item, collect results.

```
defmodule Concurrent do
  def map(elements, fun) do
    # Fork: spawn one process per element
    pids = Enum.map(elements, fn e ->
      parent = self()
      spawn(fn ->
        send(parent, {self(), fun.(e)})
      end)
    end)
    # Join: collect results in order
    Enum.map(pids, fn pid ->
      receive do
        {^pid, result} -> result
      end
    end)
  end

  Concurrent.map(ns, &Fibonacci.index/1)
```

Primitive concurrency primitives

```
# Spawn a process:
pid = spawn(fn -> ... end)

# Send a message:
send(pid, {:hello, 42})

# Receive a message (blocking):
receive do
  {:hello, x} -> x
other -> IO.inspect(other)
end

# Get own PID:
self()

# Pin operator ^ (match literal):
x = 5
receive do
  {^x, result} -> result
# only matches tuples where
# first element == 5
end
```

The fork-join pattern: spawn n processes (fork), each computes one result and sends it back; the parent collects all results (join). The ^ pin operator ensures we match the response from the correct PID.

Module 10: Concurrency (GenServer, Registry, Supervision)

Advanced path

The bakery example: **Baker** GenServers register themselves in a **Registry** by product name. **Customer** GenServers look up bakers by product and place orders asynchronously. Results arrive back via cast messages.

Baker GenServer (full pattern)

```
defmodule Baker do
  use GenServer

  # Interface
  def start(product, name) do
    GenServer.start(__MODULE__,
      [product: product, name: name])
  end

  def order(product) do
    case Registry.lookup(
      BakerRegistry, product) do
      [] ->
        {:error, "No baker"}
      workers ->
        {pid, _} = Enum.random(workers)
        GenServer.cast(pid,
          {:order, self()})
        :ok
    end
  end

  # Callbacks
  @impl true
  def init([product: p, name: _]=state) do
    Registry.register(BakerRegistry,
      p, nil)
    {:ok, state}
  end

  @impl true
  def handle_cast({:order, client},
    [product: p, name: n]=state) do
    :timer.sleep(Enum.random(1600..3200))
    GenServer.cast(client, {:cake, p, n})
    {:noreply, state}
  end
end
```

Registry

```
# Start a registry:
{:ok, _} = Registry.start_link(
  name: BakerRegistry,
  keys: :duplicate # or :unique
)

# Register self:
Registry.register(BakerRegistry,
  :bread, nil)

# Lookup all with key:
Registry.lookup(BakerRegistry, :bread)
# => [{pid1, nil}, {pid2, nil}]
```

GenServer return values

```
# init:
{:ok, initial_state}
{:stop, reason}

# handle_call:
{:reply, reply, new_state}
{:noreply, new_state}

# handle_cast:
{:noreply, new_state}
{:stop, reason, new_state}
```

Registry modes

```
# :unique → one pid per key
# :duplicate → many pids per key
# :partition → sharded (advanced)
```

Module 10 exercise: complete the Customer GenServer that places orders with random bakers and collects cakes. Uses `handle_cast` for async baked-goods delivery.

Solved Exercises — SDU IoT

Complete solutions to all exercise sets | Exam: June 9, 2026

L2 — Drinking the Elixir: Solved Exercises

Quest 0: Install LiveBook

5XP

Solution: Install Erlang → Elixir → LiveBook following livebook.dev/install. Access at <http://localhost:4000/>. Configure: Settings → User Settings → enable 'Render Ligatures' and disable 'Automatically close brackets'.

Quest 1: Kickstart

10XP

Solution: Clone <https://github.com/aslakjohansen/elixir-intro>, open `start.livemd` in LiveBook, work through modules 1–10. Beginner path: M1→M2→...→M10. Advanced path: M8→M9→M10.

Quest 2: Getting Acquainted

15XP

Solution: Clone <https://github.com/aslakjohansen/livebook-demos> and open each of the following in LiveBook, evaluating each cell in order:

- `basic-data-types.livemd` — integers, floats, atoms, strings, lists, tuples, maps
- `control-structures.livemd` — `if/else`, `case`, `cond`, `with`
- `pattern-matching.livemd` — `=` operator, destructuring, function heads
- `pipelines.livemd` — `|>` operator, function composition
- `primitive-concurrency.livemd` — `spawn`, `send`, `receive`, `self()`

The key cross-cutting dependency: **pattern matching** is used everywhere — in variable binding, function heads, `case` arms, and `receive` blocks.

Quest 3: Understanding the LiveMD Format

5XP

Answer: LiveMD files are valid **Markdown (.md)** files. The file format uses standard Markdown headings (`##`), paragraphs, and fenced code blocks. Elixir code cells use ````elixir` fences; LiveBook-specific metadata is stored in HTML comments (`<!--`). Any Markdown renderer will display them readably even without LiveBook.

Quest 4: Sources of Information

5XP

Answers:

- **Live-coding videos:** José Valim's YouTube channel — https://www.youtube.com/watch?v=mDxJqX5-ns&list=PLNP8vc86_-SOV1ZEvX_q9BLYWL586zWnF
- **Module documentation:** <https://hexdocs.pm> — e.g. hexdocs.pm/elixir/Enum.html. Also accessible in LiveBook by hovering over a function name after evaluating the cell.
- **Language concepts guide:** <https://elixir-lang.org/getting-started/introduction.html>

Quest 5: Experimenting with Pipelines

10XP

Question 1: To get a list instead of a `MapSet`, remove the `MapSet.new()` step and add `Enum.uniq()` instead:

```
# Original (returns MapSet):
input
|> String.downcase()
|> String.split()
|> Enum.map(&String.length/1)
|> Enum.filter(&(&1 > 2))
|> MapSet.new() # ← remove this

# Modified (returns list with unique values):
input
|> String.downcase()
|> String.split()
|> Enum.map(&String.length/1)
|> Enum.filter(&(&1 > 2))
|> Enum.uniq() # ← add this
# Or simply Enum.to_list() if you want all (including duplicates)
```

Question 2 — disabling String.downcase(): Without downcase, pattern matching becomes case-sensitive. Words like 'The' and 'the' are treated as different strings, producing different length values or different MapSet members. The result set changes because capitalised words are processed differently from lowercase ones.

Quest 6: Working with Functions

5XP

Answers to plotting-functions.livemd:

- **What the code does:** Generates x values in a range using a comprehension (for x <- start..stop), computes f(x) for each, packages into maps, and plots with VegaLite.
- **Comprehensions:** for x <- 1..10, do: x*2 produces [2,4,6,...20]. The do: clause is applied to each element of the generator.
- **Fill area below curve:** Replace Vl.mark(:point) with Vl.mark(:area) to shade the region between the curve and the x-axis.
- **interval variable:** Controls the update frequency of the live plot in milliseconds.
- **2nd arg to Kino.VegaLite.listen:** The event name to subscribe to (e.g. :stream_data).
- **Return value of fun:** A list of data point maps — this is appended to the chart.
- **Kino.VegaLite.push:** Appends new data points to a live chart.
- **window option:** Maximum number of data points shown — older points are discarded as new ones arrive, creating a sliding window.

Quest 7: Servers Everywhere!

10XP

Answers to generic-servers.livemd:

- **Why 'callback'?** These functions are called *by the framework* (GenServer) when a specific event occurs — inversion of control. You define what happens; the framework decides when to call it.
- **Why both call and cast?** call = synchronous (client waits for response, needed when you need a return value). cast = async (fire-and-forget, needed for one-way updates where blocking would cause deadlock or waste time).
- **Only calls?** The server becomes a bottleneck — every client blocks waiting. If two GenServers call each other in a cycle, deadlock occurs.
- **Supervision tree:** Kino.Process.render_sup_tree(sup_pid) renders a visual tree. The dynamic supervisor appears as the root; child processes as leaves.
- **Registry modes:** :unique (one PID per key), :duplicate (many PIDs per key), :partition (sharded for high-concurrency use cases).

Quest 8: World of Zuul — Fix 'say'

20XP

The 'say' command only reaches the speaking player themselves. Fix: broadcast to all players in the same space.

```

# Option A: Space tracks its players
# Add to Space GenServer state: a list of player pids/client_ids

defmodule Space do
  use GenServer

  # Add to init state:
  def init(%{title: title, exits: exits}) do
    {:ok, %{title: title, exits: exits, players: []}}
  end

  # When player enters a space:
  def handle_cast({:enter, client_id}, state) do
    {:noreply, %{state | players: [client_id | state.players]}}
  end

  # When player leaves a space:
  def handle_cast({:leave, client_id}, state) do
    {:noreply, %{state | players: List.delete(state.players, client_id)}}
  end

  # Broadcast 'say' to all players in the space:
  def handle_cast({:say, from_id, message}, state) do
    Enum.each(state.players, fn client_id ->
      # Append to each player's Kino frame
      Player.receive_message(client_id, from_id, message)
    end)
    {:noreply, state}
  end

  # In the command handler for 'say':
  # 1. Look up which space the current player is in
  # 2. Call Space.say(space_pid, client_id, message)
  # Update 'go' to call Space.leave(old_space, id) and Space.enter(new_space, id)

```

Key: maintain a player→space mapping (either in the Space server or in a separate Registry). Update it on every 'go' command. On 'say', look up all players in the current space and deliver to each.

L3 — Metadata: Adventures in Semantic Madness: Solved Exercises

Quest 1: Exploring the Lecture Example

5XP

Solution: Study the lecture building model in `rdf.livemd` or `neo4j.livemd`. The model contains: a building with floors, rooms on each floor, and data streams (sensors) attached to rooms. The 'crossed streams' problem means some sensors may be connected to wrong rooms in the model — understanding the hierarchy is essential before writing queries that traverse it.

Quest 2: Exploring the Source

5XP

Strategy: Breadth-first (same exercise in both models before moving on) reveals the comparison most clearly — you immediately see how the same concept (e.g. room containment) is expressed differently in RDF triples vs Neo4j nodes/edges.

Quest 3: A New Query — Rooms with Kelvin streams

5XP

SPARQL solution:

```

PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX unit: <http://qudt.org/vocab/unit/>

SELECT DISTINCT ?room_name WHERE {
  ?building a brick:Building .
  ?building brick:hasPart ?room .
  ?room a brick:Room .
  ?room rdfs:label ?room_name .
  ?stream brick:pointOf ?room .
  ?stream brick:hasUnit unit:K .
}

```

Cypher solution:

```

MATCH (b:Building)-[:hasPart]->(r:Room)
<-[:pointOf]-(s:Stream)
WHERE s.unit = 'K'
RETURN DISTINCT r.label AS room_name

```

Quest 4: Universe Expansion — Add Building type**10XP****RDF/Turtle solution:**

```

@prefix brick: <https://brickschema.org/schema/Brick#> .
@prefix demo: <http://ontologies.sdu.dk/Demo#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

# 1. Building type already in Brick schema.
# 2. Add instance and connect to floors:
demo:SDU_Building a brick:Building ;
rdfs:label "SDU Building" ;
brick:hasPart demo:floor1 ;
brick:hasPart demo:floor2 .

# 3. Verify query:
SELECT ?floor WHERE {
  demo:SDU_Building brick:hasPart ?floor .
  ?floor a brick:Floor .
}

```

Cypher solution:

```

// Create building node:
CREATE (b:Building {
  label: 'SDU Building'
})

// Connect to existing floors:
MATCH (b:Building {label:'SDU Building'}),
(f:Floor)
CREATE (b)-[:hasPart]->(f)

// Verify:
MATCH (b:Building)-[:hasPart]->(f:Floor)
RETURN b.label, f.label

```

Quest 5: Temperature Actuator — Thermostat model**15XP****Design decisions:**

- Use brick:pointOf for ALL data points (sensor, setpoint, actuator) — consistent relationship
- brick:TemperatureSetpoint already in Brick schema; use brick:Valve_Position for actuator
- Missing one of the three → room produces NO ROW in result (SQL-style inner join semantics)

RDF — add instances:

```

demo:room1_setpoint a brick:Temperature_Setpoint ;
rdfs:label "room1-setpoint-uuid" ;
brick:pointOf demo:room1 .

demo:room1_valve a brick:Radiator_Valve_Position ;
rdfs:label "room1-valve-uuid" ;
brick:pointOf demo:room1 .

```

Cypher thermostat query:

```

MATCH (r:Room)
<-[:pointOf]-(s:Temperature_Sensor),
(r)<-[:pointOf]-(sp:Temperature_Setpoint),
(r)<-[:pointOf]-(a:Valve_Position)
RETURN r.label AS room,
s.uuid AS sensor_uuid,
sp.uuid AS setpoint_uuid,
a.uuid AS actuator_uuid

```

SPARQL thermostat query:

```

SELECT ?room ?s_uuid ?sp_uuid ?a_uuid WHERE {
  ?room a/brick:subClassOf* brick:Room .
  ?sensor a/subClassOf* brick:Temperature_Sensor ;
  rdfs:label ?s_uuid .
  ?setpoint a/subClassOf* brick:Temperature_Setpoint ;
  rdfs:label ?sp_uuid .
  ?actuator a/subClassOf* brick:Radiator_Valve_Position ;
  brick:pointOf ?room ;
  rdfs:label ?a_uuid .
}

```

Missing component → that room is excluded from the result set entirely. This is correct for a thermostat app: you only want rooms where ALL three components exist. Use LEFT JOIN / OPTIONAL if you want to see partial rooms.

Quest 6: Schema / Model Checker**20XP**

1. What would a model checker look like?

A set of queries that find *violations* — nodes/triples that break the model rules. Each query returns a (hopefully empty) result set. Non-empty = violation found.

2. Missing rules:

- Every brick:pointOf must have Equipment as subject and Location as object
- Every Temperature_Sensor must have exactly one brick:unit
- Every Room used in a thermostat query must have all three components
- No circular containment (A hasPart B hasPart A)

3. Describe them (SHACL for RDF):

```
# SHACL shape: pointOf must link Equipment → Location
demo:PointOfShape a sh:PropertyShape ;
sh:path brick:pointOf ;
sh:class brick:Location ;
sh:nodeKind sh:IRI .
```

```
# Negative query as model checker (SPARQL):
# Find rooms missing a temperature sensor:
SELECT ?room WHERE {
?room a/brick:subClassOf* brick:Room .
FILTER NOT EXISTS {
?s a/brick:subClassOf* brick:Temperature_Sensor ;
brick:pointOf ?room .
}
}
```

```
# Cypher equivalent:
MATCH (r:Room)
WHERE NOT (r)-[:pointOf]-(:Temperature_Sensor)
RETURN r.label AS room_missing_sensor
```

L4 — C Exercises: Into the Muck: Solved Exercises

Quest 1: Making that First Compile

5XP

```
/* hello.c */
#include <stdio.h>
int main(int argc, char* argv[]) {
printf("Hello, World\n");
return 0;
}
```

```
# Compile and run:
$ gcc hello.c -o hello
$ ./hello
Hello, World
```

Quest 2: Offsetting the Index

10XP


```

/* index1.c – complete solution */
#include <stdio.h>
#include <stdlib.h>
#define SIZE 10

int main(int argc, char* argv[]) {
/* Step 4: allocate */
int* a = (int*) malloc(SIZE * sizeof(int));

/* Step 8: initialise a[i] = i */
for (int i = 0; i < SIZE; i++)
a[i] = i;

/* Step 11-12: b is 1-indexed alias of a */
int* b = &a[0]; /* b points to a[0] */
b--; /* b now points to a[-1] logically */
/* so b[1]==a[0], b[10]==a[9] */

/* Step 13: loop from 1 to SIZE (inclusive) */
for (int i = 1; i <= SIZE; i++)
printf("%d: %d\n", i, b[i]);

/* Step 15: b[0] is one int BEFORE a[0] – garbage/UB */
printf("b[0] = %d (undefined behaviour!)\n", b[0]);
free(a);
return 0;
}

```

Step 6 explanation: Uninitialized malloc memory contains whatever was previously in that heap region — garbage values. Not zero.

Step 16 explanation: `b[0]` reads one int *before* `a[0]` — outside the allocation. This is undefined behaviour. In practice it reads a heap bookkeeping word or random memory. The loop change: initialization `i=1`, condition `i≤SIZE`, step `i++`.

Quest 3: In Support of a Hero — TSP

30XP

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define CITY_COUNT (15)

char* citynames[CITY_COUNT] = {
    "Esbjerg", "Helsingør", "Herning", "Horsens", "Kolding",
    "København", "Næstved", "Odense", "Randers", "Roskilde",
    "Silkeborg", "Vejle", "Viborg", "Aalborg", "Århus"
};

int distances[CITY_COUNT][CITY_COUNT] = {
    { 0, 269, 82, 98, 65, 260, 211, 123, 149, 230, 105, 74, 126, 198, 134},
    {269, 0, 226, 173, 206, 40, 105, 157, 166, 55, 191, 196, 207, 200, 150},
    { 82, 226, 0, 63, 79, 230, 202, 121, 76, 202, 36, 59, 45, 117, 77},
    { 98, 173, 63, 0, 48, 172, 139, 62, 68, 142, 40, 26, 75, 132, 40},
    { 65, 206, 79, 48, 0, 196, 148, 59, 114, 165, 77, 25, 110, 176, 88},
    {260, 40, 230, 172, 196, 0, 71, 141, 180, 31, 196, 190, 218, 224, 156},
    {211, 105, 202, 139, 148, 71, 0, 89, 174, 50, 175, 150, 204, 232, 142},
    {123, 157, 121, 62, 59, 141, 89, 0, 121, 110, 102, 64, 136, 186, 86},
    {149, 166, 76, 68, 114, 180, 174, 121, 0, 156, 44, 90, 42, 65, 36},
    {230, 55, 202, 142, 165, 31, 50, 110, 156, 0, 169, 160, 193, 206, 130},
    {105, 191, 36, 40, 77, 196, 175, 102, 44, 169, 0, 53, 35, 99, 41},
    { 74, 196, 59, 26, 25, 190, 150, 64, 90, 160, 53, 0, 86, 151, 65},
    {126, 207, 45, 75, 110, 218, 204, 136, 42, 193, 35, 86, 0, 72, 63},
    {198, 200, 117, 132, 176, 224, 232, 186, 65, 206, 99, 151, 72, 0, 101},
    {134, 150, 77, 40, 88, 156, 142, 86, 36, 130, 41, 65, 63, 101, 0}
};

int main(void) {
    int visited[CITY_COUNT];
    int route[CITY_COUNT + 1];
    memset(visited, 0, sizeof(visited));

    int current = 0; /* Start at Esbjerg */
    visited[current] = 1;
    route[0] = current;
    int total = 0;

    /* Nearest-neighbour greedy heuristic */
    for (int step = 1; step < CITY_COUNT; step++) {
        int best = -1, best_dist = 1<<30;
        for (int j = 0; j < CITY_COUNT; j++) {
            if (!visited[j] && distances[current][j] < best_dist) {
                best_dist = distances[current][j];
                best = j;
            }
        }
        visited[best] = 1;
        route[step] = best;
        total += best_dist;
        current = best;
    }
    /* Return to start */
    total += distances[current][route[0]];
    route[CITY_COUNT] = route[0];

    /* Print route */
    printf("Route: ");
    for (int i = 0; i <= CITY_COUNT; i++)
        printf("%s%s", citynames[route[i]],
            i < CITY_COUNT ? " -> " : "\n");
    printf("Total distance: %d km\n", total);
    return 0;
}

```

Expected output (nearest-neighbour):

Route: Esbjerg → Kolding → Vejle → Horsens → Silkeborg → Herning → Viborg → Aalborg → Randers → Århus → Roskilde → København → Helsingør → Næstved → Odense → Esbjerg
 Total distance: approximately 1,240 km

Note: Nearest-neighbour is a heuristic — not optimal. The optimal TSP solution for these 15 cities is shorter but requires exponentially more computation.

The mystery code explained:

Setup: allocates 12 ints (each 4 bytes) = 48 bytes total. Fills them: values[0]=0, values[1]=1, ..., values[11]=11.

Then it **reinterprets the same memory as long***. On a 64-bit system sizeof(long)=8, so each long spans TWO consecutive ints.

On a **little-endian** 64-bit system:

- ((long*)values)[0] reads bytes of values[0] and values[1] together
- values[0]=0 (0x00000000) stored at low address, values[1]=1 (0x00000001) at high
- As a 64-bit little-endian long: 0x00000001_00000000 = 4294967296
- ((long*)values)[1] = values[2]+values[3] = 0x00000003_00000002 = 12884901890

The loop runs LENGTH/2 = 6 times, printing pairs:

- i=0: ((long*)values)[0] = $1 \times 2^{32} + 0 = 4294967296$
- i=1: ((long*)values)[1] = $3 \times 2^{32} + 2 = 12884901890$
- i=2: $5 \times 2^{32} + 4 = 21474836484$
- i=3: $7 \times 2^{32} + 6 = 30064771078$
- i=4: $9 \times 2^{32} + 8 = 38654705672$
- i=5: $11 \times 2^{32} + 10 = 47244640266$

Why it seems mysterious: The code prints huge numbers from small ones. It demonstrates pointer aliasing, type punning, and platform-dependent behaviour. The (int) cast in printf truncates the result — so the printed values may appear negative or unexpected depending on the platform.

L5 — Toolchain: A Device Takes Its First Step: Solved Exercises

Quest 2: Blinking Away — blink2 with parameters

10XP

Key extension: pass delay as parameter to reuse TaskBlink for multiple LEDs.

```
#include <stdio.h>
#include "sdkconfig.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

typedef struct {
    gpio_num_t pin;
    uint32_t delay_ms;
} blink_params_t;

void TaskBlink(void* pvParameters) {
    blink_params_t* params = (blink_params_t*) pvParameters;
    uint8_t state = 1;
    while (1) {
        gpio_set_level(params->pin, state);
        vTaskDelay(params->delay_ms / portTICK_PERIOD_MS);
        state = !state;
    }
}

static blink_params_t p1 = {.pin = 33, .delay_ms = 1000};
static blink_params_t p2 = {.pin = 32, .delay_ms = 300};

void app_main(void) {
    gpio_reset_pin(33); gpio_set_direction(33, GPIO_MODE_OUTPUT);
    gpio_reset_pin(32); gpio_set_direction(32, GPIO_MODE_OUTPUT);
    xTaskCreate(TaskBlink, "Blink1", 2048, &p1, 1, NULL);
    xTaskCreate(TaskBlink, "Blink2", 2048, &p2, 1, NULL);
}
```

Quest 3: Serial Count

15XP

```

#include <stdio.h>
#include <stdlib.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/uart.h"

#define UART_NUM UART_NUM_0
#define BUF_SIZE 256

void uart_init(void) {
    uart_config_t cfg = {
        .baud_rate = 115200,
        .data_bits = UART_DATA_8_BITS,
        .parity = UART_PARITY_DISABLE,
        .stop_bits = UART_STOP_BITS_1,
        .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
    };
    uart_driver_install(UART_NUM, BUF_SIZE, 0, 0, NULL, 0);
    uart_param_config(UART_NUM, &cfg);
}

void count_down(uint8_t n) {
    char buf[8];
    for (int i = n; i >= 0; i--) {
        snprintf(buf, sizeof(buf), "%d\n", i);
        uart_write_bytes(UART_NUM, buf, strlen(buf));
    }
}

void app_main(void) {
    uart_init();
    char line[BUF_SIZE];
    while (1) {
        /* Read line from UART */
        int len = uart_read_bytes(UART_NUM, (uint8_t*)line,
            BUF_SIZE-1, pdMS_TO_TICKS(100));
        if (len > 0) {
            line[len] = '\0';
            int n = atoi(line);
            count_down((uint8_t) n);
        }
    }
}

```

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "driver/uart.h"
#include "driver/gpio.h"

#define UART_NUM UART_NUM_0
#define BUF_SIZE 256
#define MAX_ARGS 16
#define COMMAND_COUNT 5

typedef void (*command_t)(int argc, char* argv[]);
typedef struct { char* name; command_t fn; } dispatch_entry_t;

void command_echo(int argc, char* argv[]) {
    for (int i = 1; i < argc; i++) {
        uart_write_bytes(UART_NUM, argv[i], strlen(argv[i]));
        uart_write_bytes(UART_NUM, " ", 1);
    }
    uart_write_bytes(UART_NUM, "\n", 1);
}

void command_pin_high(int argc, char* argv[]) {
    if (argc < 2) return;
    int pin = atoi(argv[1]);
    gpio_reset_pin(pin);
    gpio_set_direction(pin, GPIO_MODE_OUTPUT);
    gpio_set_level(pin, 1);
}

void command_pin_low(int argc, char* argv[]) {
    if (argc < 2) return;
    int pin = atoi(argv[1]);
    gpio_reset_pin(pin);
    gpio_set_direction(pin, GPIO_MODE_OUTPUT);
    gpio_set_level(pin, 0);
}

void command_pin_read(int argc, char* argv[]) {
    if (argc < 2) return;
    int pin = atoi(argv[1]);
    gpio_reset_pin(pin);
    gpio_set_direction(pin, GPIO_MODE_INPUT);
    char buf[4];
    sprintf(buf, sizeof(buf), "%d\n", gpio_get_level(pin));
    uart_write_bytes(UART_NUM, buf, strlen(buf));
}

void command_reverse(int argc, char* argv[]) {
    for (int i = argc-1; i >= 1; i--) {
        uart_write_bytes(UART_NUM, argv[i], strlen(argv[i]));
        uart_write_bytes(UART_NUM, " ", 1);
    }
    uart_write_bytes(UART_NUM, "\n", 1);
}

dispatch_entry_t command_dispatch_table[COMMAND_COUNT] = {
    {"echo", command_echo},
    {"pin-high", command_pin_high},
    {"pin-low", command_pin_low},
    {"pin-read", command_pin_read},
    {"reverse", command_reverse},
};

```

cnc.c continued — dispatch function and main loop:

```

void dispatch(char* input) {
    char* args[MAX_ARGS];
    int argc = 0;
    char* p = input;
    /* Split on spaces by inserting null bytes */
    while (*p && argc < MAX_ARGS) {
        while (*p == ' ') p++;
        if (*p == '\0' || *p == '\n') break;
        args[argc++] = p;
        while (*p && *p != ' ' && *p != '\n') p++;
        if (*p) *p++ = '\0';
    }
    if (argc == 0) return;
    for (int i = 0; i < COMMAND_COUNT; i++) {
        if (strcmp(args[0], command_dispatch_table[i].name) == 0) {
            command_dispatch_table[i].fn(argc, args);
            return;
        }
    }
    uart_write_bytes(UART_NUM,
        "Unknown command. I regret being unable to comply.\n", 48);
}

void app_main(void) {
    /* uart_init() as in Quest 3 */
    char line[BUF_SIZE];
    while (1) {
        int len = uart_read_bytes(UART_NUM, (uint8_t*)line,
            BUF_SIZE-1, pdMS_TO_TICKS(100));
        if (len > 0) { line[len] = '\0'; dispatch(line); }
    }
}

```

L6 — Sampling Physical Phenomena: Solved Exercises

Quest 1: Seeing Potential — ADC sample()

15XP

```

#include <stdio.h>
#include "esp_adc/adc_oneshot.h"

#define ADC_CHANNEL ADC_CHANNEL_6 /* GPIO34 on ESP32-DevKitC */

static adc_oneshot_unit_handle_t adc_handle;

void adc_init(void) {
    adc_oneshot_unit_init_cfg_t unit_cfg = {.unit_id = ADC_UNIT_1};
    adc_oneshot_new_unit(&unit_cfg, &adc_handle);
    adc_oneshot_chan_cfg_t chan_cfg = {
        .atten = ADC_ATTEN_DB_11, /* 0-3.3V range */
        .bitwidth = ADC_BITWIDTH_12, /* 0-4095 */
    };
    adc_oneshot_config_channel(adc_handle, ADC_CHANNEL, &chan_cfg);
}

int sample(void) {
    int raw;
    adc_oneshot_read(adc_handle, ADC_CHANNEL, &raw);
    return raw;
}

void app_main(void) {
    adc_init();
    while (1) {
        printf("%d\n", sample());
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

```

Quest 3: Getting Real — LMT86 temperature conversion

10XP

```

/* convert: raw ADC value → temperature in °C */
/* LMT86: Vout = -13.582 mV/°C + 2068.3 mV at 0°C (approx) */
/* Exact from §8.3.1: T = (Vmv - 2068.3) / (-13.582) */
/* ESP32 ADC: 12-bit, Vref ≈ 1100 mV with DB_11 atten (0-3.3V) */

int8_t convert(int raw) {
/* Step 1: raw → millivolts */
/* With DB_11: full scale 3300 mV, 12-bit = 4095 counts */
int mv = (raw * 3300) / 4095;

/* Step 2: millivolts → °C using LMT86 linear approx */
/* T = (mv - 2068.3) / (-13.582) */
/* Integer arithmetic: multiply by 10 to keep 1 decimal */
/* T*10 = (mv*10 - 20683) / (-13582/1000) */
/* Simplified integer: T ≈ (2068 - mv) * 1000 / 13582 */
int temp_c = ((2068 - mv) * 1000) / 13582;
return (int8_t) temp_c;
}

/* Note: LMT86 V decreases as temperature increases. */
/* Higher temperature → lower ADC raw → higher Vout in range */

```

Quest 6: Laying Pipes — FreeRTOS queue pipeline

10XP

Full pipelined-sample-transmit solution:

```

#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "esp_adc/adc_oneshot.h"

#define TX_BUFFER_SIZE 10
#define ADC_CHANNEL ADC_CHANNEL_6

static adc_oneshot_unit_handle_t adc_handle;
QueueHandle_t tx_queue;

void adc_init(void) { /* same as Quest 1 */ }

void TaskSample(void* pvParameters) {
QueueHandle_t output_queue = (QueueHandle_t) pvParameters;
while (1) {
int value;
adc_oneshot_read(adc_handle, ADC_CHANNEL, &value);
while (xQueueSendToBack(output_queue, &value, 10) != pdTRUE);
}
}

void TaskTransmit(void* pvParameters) {
QueueHandle_t input_queue = (QueueHandle_t) pvParameters;
int value;
while (1) {
while (xQueueReceive(input_queue, &value, 10) != pdPASS);
printf("%d\n", value);
}
}

void app_main(void) {
adc_init();
tx_queue = xQueueCreate(TX_BUFFER_SIZE, sizeof(int));
xTaskCreate(TaskSample, "Sample", 4096, tx_queue, 1, NULL);
xTaskCreate(TaskTransmit, "Transmit", 4096, tx_queue, 1, NULL);
}

```

L7 — Radio Exercises: Solved Exercises

Quest 2: Obtaining an IP Address — hl_wifi module

10XP

How SSID/password reach sdkconfig: The 'idf.py menuconfig' command reads Kconfig.projbuild files from all components. The user fills in values in the TUI. These are saved to sdkconfig and exposed as #defines (CONFIG_EXAMPLE_WIFI_SSID, etc.) which are included via sdkconfig.h.

How Kconfig.projbuild works: Defines CONFIG_ entries that appear in menuconfig. These propagate to sdkconfig.h and can be used with #ifdef in C code.

```
/* hl_wifi.h */
#ifndef __HL_WIFI_H
#define __HL_WIFI_H
#include <stdint.h>
#include <netinet/in.h>

typedef void (*connect_callback_t)(void);
typedef struct sockaddr_in sockaddr_in_t;

void hl_wifi_init(connect_callback_t* cb);
sockaddr_in_t hl_wifi_make_addr(char* ip, uint16_t port);
int hl_wifi_tcp_connect(sockaddr_in_t addr);
void hl_wifi_tcp_tx(int sock, void* buf, uint16_t len);
#endif
```

Quest 3: Uplink — complete hl_wifi implementation

15XP


```

/* hl_wifi.c - key functions */
#include "hl_wifi.h"
#include <sys/socket.h>
#include <arpa/inet.h>
#include <string.h>
#include <errno.h>
#include <stdio.h>

sockaddr_in_t hl_wifi_make_addr(char* ip, uint16_t port) {
sockaddr_in_t addr;
addr.sin_addr.s_addr = inet_addr(ip);
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
return addr;
}

int hl_wifi_tcp_connect(sockaddr_in_t addr) {
int sock = socket(AF_INET, SOCK_STREAM, IPPROTO_IP);
if (sock < 0) {
printf("socket() failed: errno=%d\n", errno);
return -1;
}
int err = connect(sock, (struct sockaddr*)&addr,
sizeof(struct sockaddr_in6));
if (err != 0) {
printf("connect() failed: errno=%d\n", errno);
return -1;
}
return sock;
}

void hl_wifi_tcp_tx(int sock, void* buf, uint16_t len) {
uint16_t offset = 0;
while (offset < len) {
uint16_t remainder = len - offset;
int sent = send(sock, (char*)buf + offset, remainder, 0);
if (sent < 0) {
printf("send() error: errno=%d\n", errno);
return;
}
offset += sent;
}
}

/* In main.c - TaskCount with counter */
void TaskCount(void* pvParameters) {
sockaddr_in_t addr = hl_wifi_make_addr("192.168.1.100", 8000);
int sock = hl_wifi_tcp_connect(addr);
if (sock == -1) { vTaskDelete(NULL); return; }

uint16_t counter = 1;
char buffer[7]; /* max 5 digits + \n + \0 = 7 */
while (1) {
sprintf(buffer, "%u\n", counter);
hl_wifi_tcp_tx(sock, buffer, strlen(buffer));

counter++;
vTaskDelay(pdMS_TO_TICKS(1000));
}

/* buffer[7] answer: max uint16_t = 65535 (5 chars) + '\n' + '\0' = 7 */
/* sprintf(%u, counter) formats unsigned int into the buffer */

```

L8 — Stream Processing: Solved Exercises

Quest 1: Installing MQTT Broker

5XP

Install and test Mosquitto:

```
$ sudo apt-get install mosquitto mosquitto-clients
# Terminal 1:
$ mosquitto_sub -t mytopic
# Terminal 2:
$ mosquitto_pub -t mytopic -m "hello, world"
# Terminal 1 should show: hello, world
```

Quest 2: Signal Generator — answers

10XP

- 1. What you see:** Sine-wave-like numeric values published to MQTT topics (e.g. siggen/room1/temp, siggen/room1/rhum). Values change smoothly over time with noise.
- 2. Concurrency:** Each room's generator runs as a separate BEAM process spawned with spawn/1 or Task.async. Processes send independently to MQTT.
- 3. Process count:** Given n configurations → **n processes** (one per room stream) + 1 MQTT client process + 1 supervisor = n+2 total.
- 4. Process diagram:**
 - Config list → [spawn] → Process 1 (room1/temp): timer → generate → MQTT publish
 - Config list → [spawn] → Process 2 (room1/rhum): timer → generate → MQTT publish
 - ... × n rooms × 2 streams = 2n processes → MQTT broker (queue) → subscribers

Quest 3: Moving Averages — answers

10XP

- 1. What you see:** Smoothed values on mavg/* topics — less noise than siggen/* but same trend. The moving average lags slightly behind rapid changes.
- 2. The window:** The last N values used to compute the average (sliding buffer).
- 3. Unit of values in window:** Same unit as input — raw sensor values (°C or %).
- 4. Lost message consequence:** The window retains the old value at that slot until it's overwritten N messages later. The average is slightly biased toward the old value.
- 5. Concurrency:** One BEAM process per room stream subscribes, maintains the window in its state, and publishes to the mavg/* topic.
- 6. Process count:** n configurations → **n processes** (one per stream).
- 7. Diagram:** siggen/* → MQTT broker → mavg process (window state) → MQTT → mavg/*

Quest 4: Absolute Humidity — Elixir solution

10XP

```

# Absolute humidity formula:
# Habs = 6.112 * exp(17.67*T / (T+243.5)) * Hrel * 2.1674 / (273.15+T)

defmodule AbsHumidity do
  def compute(temp_c, rel_hum_pct) do
    6.112 *
    :math.exp(17.67 * temp_c / (temp_c + 243.5)) *
    rel_hum_pct *
    2.1674 / (273.15 + temp_c)
  end
end

# GenServer that subscribes to temp and rhum, publishes ahum:
defmodule AhumWorker do
  use GenServer

  def start_link(room) do
    GenServer.start_link(__MODULE__, room)
  end

  def init(room) do
    # Subscribe to both temperature and humidity for this room
    Tortoise.subscribe(client_id(), "siggen/#{room}/temp", qos: 0)
    Tortoise.subscribe(client_id(), "siggen/#{room}/rhum", qos: 0)
    {:ok, %{room: room, temp: nil, rhum: nil}}
  end

  # Handle incoming MQTT message:
  def handle_cast({:message, "siggen/" <> rest, payload}, state) do
    val = String.to_float(payload)
    [room, type] = String.split(rest, "/")
    new_state = Map.put(state, String.to_atom(type), val)
    # Publish if both values available:
    if new_state.temp != nil and new_state.rhum != nil do
      ahum = AbsHumidity.compute(new_state.temp, new_state.rhum)
      Tortoise.publish(client_id(),
        "func/#{room}/ahum", Float.to_string(ahum), qos: 0)
    end
    {:noreply, new_state}
  end
end

# Process count: n rooms → n AhumWorker processes
# Each needs BOTH temp AND rhum → trigger condition: latest value
# Concurrency: one BEAM process per room, independent state

```

Quest 5: Topic Structure and Latency — answers

20XP

Topic naming convention:

- home/laptop/uptime → laptop uptime (seconds since boot)
- home/room1/temperature/celsius → room temperature
- home/room1/temperature/fahrenheit → converted temperature
- home/room1/humidity/relative → room humidity
- Differentiate uptime from temperature: use different subtopic (uptime vs temperature)

(a) End-to-end delay: $D = \text{publish_latency} + \text{broker_routing} + \text{subscribe_delivery} + \text{processing_time} + \text{re-publish_latency} + \text{broker_routing} + \text{client_delivery}$. Typically 5–50ms on a local broker.

(b) Adding room humidity: Add home/+ /humidity/relative. Works cleanly — the + wildcard covers any room.

(c) Adding outdoor temperature: Add home/outdoor/temperature/celsius. The Celsius→Fahrenheit converter subscribing to home/+ /temperature/celsius would also receive outdoor values — either desired or a design flaw depending on intent.

(d) Moving to friend's laptop: Latency formula adds $2 \times \text{network_rtt}$ between laptops. If laptops are on the same network: +1–5ms. Over the internet: +20–200ms. Clocks diverge → cross-machine timestamps need NTP correction.

Quest 6: Data Flows — answers

10XP

1. When to choose MQTT/GenServer over Flow:

- MQTT+GenServer: distributed systems, IoT devices on different machines, data comes from external sources, need persistence/QoS, cross-language integration
- Flow: single-machine batch/stream processing, word counts, log analysis, ETL pipelines, where all data is available locally and concurrency is the goal

2. When to combine:

- Use MQTT to receive data from IoT devices across the network
- Feed MQTT messages into a Flow pipeline for local high-throughput processing (e.g. window aggregations over thousands of messages/second)
- Then publish results back to MQTT for downstream consumers

This is the 'intake via MQTT, process via Flow, emit via MQTT' pattern — gets the best of both worlds.

Quest 7: Chatty Room — ESP32 MQTT chat

15XP

Complete main.c solution for the chat application:

```
#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "mqtt_client.h"
#include "driver/uart.h"
/* Include hl_wifi.h from radio exercise */

#define BROKER_URL "mqtt://broker.hivemq.com"
#define TOPIC "org/sdu/course/iot/year/2026/chat/channel/42"
#define UART_NUM UART_NUM_0
#define BUF_SIZE 256

char nick[10];
esp_mqtt_client_handle_t mqtt_client;

/* TaskChat: read serial line, publish to MQTT */
void TaskChat(void* pvParameters) {
    char line[BUF_SIZE];
    char buffer[BUF_SIZE];
    while (1) {
        int len = uart_read_bytes(UART_NUM, (uint8_t*)line,
            BUF_SIZE-1, pdMS_TO_TICKS(100));
        if (len > 0) {
            line[len] = '\0';
            /* Remove trailing newline */
            if (line[len-1] == '\n') line[len-1] = '\0';
            sprintf(buffer, "%s: %s\n", nick, line);
            esp_mqtt_client_publish(mqtt_client, TOPIC,
                buffer, 0, 1, 0);
        }
    }
}

static void mqtt_event_handler(void* handler_args,
    esp_event_base_t base, int32_t event_id, void* event_data) {
    esp_mqtt_event_handle_t event = event_data;
    esp_mqtt_client_handle_t client = event->client;
    switch ((esp_mqtt_event_id_t)event_id) {
        case MQTT_EVENT_CONNECTED:
            esp_mqtt_client_subscribe(client, TOPIC, 1);
            xTaskCreate(TaskChat, "Chat", 4096, NULL, 1, NULL);
            break;
        case MQTT_EVENT_DATA:
            printf("%.s\n", event->data_len, event->data);
            break;
        default:
            printf("MQTT event %d\n", event_id);
    }
}

void connected_callback(void) {
    /* Prompt for nickname */
    printf("Enter nickname (max 9 chars): ");
    uart_read_bytes(UART_NUM, (uint8_t*)nick, 9, portMAX_DELAY);
}
```

```

nick[9] = '\0';
/* Remove newline */
for (int i=0; i<10; i++) if (nick[i]!='\n') { nick[i]='\0'; break; }

esp_mqtt_client_config_t mqtt_cfg = {.uri = BROKER_URL};
mqtt_client = esp_mqtt_client_init(&mqtt_cfg);
esp_mqtt_client_register_event(mqtt_client, ESP_EVENT_ANY_ID,
mqtt_event_handler, NULL);
esp_mqtt_client_start(mqtt_client);
}

/* nick is char[10]: max 9 printable chars + null terminator.
Longer nicknames are silently truncated. */

```

L9 — Performance Evaluation: Solved Exercises

Quest 1: Test Harness — answers

10XP

1. How to instrument an arbitrary command: Wrap the command in the test harness function which captures start time, executes the command, captures end time, and records memory/CPU metrics. In the BEAM: use `:timer.tc/1` to measure execution time in microseconds; use `Process.info(pid)` to get memory usage.

```

{time_us, result} = :timer.tc(fn -> your_command() end)
# time_us = microseconds elapsed
mem = Process.info(self(), :memory)

```

2. If the process starts a new process: The harness measures only the parent process. Child processes run concurrently and are NOT measured unless you explicitly instrument them too. This is analogous to `fork()` — the harness sees only its own PID's stats.

3. How to read resulting data: The test harness writes to a log file or returns a list of measurements. Parse the file with pandas (Python) or `Enum.reduce` (Elixir). Group by factor value, compute statistics (mean, p50, p90, p99), then visualise.

Quest 3: End-to-End Latency — complete methodology

30XP

Question	Answer
1. End-to-end latency	Time from Signal Generator publishing a value to Client receiving the corresponding absolute humidity result. Spans: siggen publish → broker → mavg process → broker → ahum process → broker → client.
2. Result form	A distribution — not a single number. Report as: histogram, CDF (easy to read p50/p90), or violin plot. Include mean, median, p90, p99 latency.
3. System definition	Components: Signal Generator, MQTT Broker (stub), Moving Average worker(s), Absolute Humidity worker(s). Stubs replace: IoT hardware (signal generator is synthetic), client (mosquitto_sub serves as client).
4. Relevant services	Service: publish one sensor reading and receive one absolute humidity result. Each such round-trip = one latency measurement.
5. Metrics	End-to-end latency [ms]; throughput [messages/s]; optionally: per-hop latency.
6. Parameters & factors	Fixed: broker type (Mosquitto), OS, machine. Factor: number of rooms $n \in \{1, 2, 4, 8\}$ — to test scaling.
7. Workload	Signal generator publishes 1 message/second per room for 60 seconds. Label each message with a unique ID and timestamp for matching.
8. Experimental design	Full-factorial over $n \in \{1, 2, 4, 8\}$. Repeat each configuration 5 times. Wait 5s between runs for cleanup. Total: $4 \times 5 = 20$ runs.
9. Instrumentation	Add timestamp T1 to each published message payload. Record T2 when client receives the matching result. Latency = $T2 - T1$. Write (msg_id, T1, T2) to log file.

10. Post-processing	Parse log CSV. Match msg_ids. Compute latency = T2-T1. Filter outliers $>3\sigma$. Group by n_rooms. Build distribution per group.
11. Visualisation	CDF plot (x=latency [ms], y=fraction of messages) with one curve per n_rooms value. Violin plot for distribution shape comparison across factor levels.
12. Expected conclusion	Latency grows with n_rooms as broker must route more messages and more processes compete. The median likely stays $< 20\text{ms}$ locally; the p99 tail grows faster with load. The system is likely MQTT broker-bound, not process-bound.

L9b — Budget Management: Solved Exercises

Quest 1: Running Average — complete pipeline

10XP

```
#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "esp_adc/adc_oneshot.h"

#define TX_BUFFER_SIZE 10
#define AVG_BUFFER_SIZE 10
#define AVG_WINDOW_SIZE 7
#define ADC_CHANNEL ADC_CHANNEL_6

typedef struct {
    QueueHandle_t input;
    QueueHandle_t output;
} stage_interface_t;

static adc_oneshot_unit_handle_t adc_handle;
QueueHandle_t tx_queue;
QueueHandle_t avg_queue;
stage_interface_t avg_pair;

void TaskSample(void* pvParameters) {
    QueueHandle_t output_queue = (QueueHandle_t) pvParameters;
    while (1) {
        int value;
        adc_oneshot_read(adc_handle, ADC_CHANNEL, &value);
        while (xQueueSendToBack(output_queue, &value, 10) != pdTRUE);
    }
}

void TaskAvg(void* pvParameters) {
    stage_interface_t* pair = (stage_interface_t*) pvParameters;
    QueueHandle_t input_queue = pair->input;
    QueueHandle_t output_queue = pair->output;

    int buffer[AVG_WINDOW_SIZE];
    memset(buffer, 0, AVG_WINDOW_SIZE * sizeof(buffer[0]));
    uint8_t index = 0;
    int sum = 0;

    /* BONUS: pre-fill AVG_WINDOW_SIZE-1 messages for stable startup */
    for (int i = 0; i < AVG_WINDOW_SIZE - 1; i++) {
        int new_val;
        while (xQueueReceive(input_queue, &new_val, portMAX_DELAY) != pdPASS);
        int old = buffer[index];
        buffer[index] = new_val;
        sum = sum - old + new_val;
        index = (index + 1) % AVG_WINDOW_SIZE;
    }

    while (1) {
        /* Receive */
        int new_val;
        while (xQueueReceive(input_queue, &new_val, 10) != pdPASS);
```

```

/* Update circular buffer */
int old = buffer[index];
buffer[index] = new_val;
sum = sum - old + new_val;
index = (index + 1) % AVG_WINDOW_SIZE;

/* Output average */
int value = sum / AVG_WINDOW_SIZE;
while (xQueueSendToBack(output_queue, &value, 10) != pdTRUE);
}
}

void TaskTransmit(void* pvParameters) {
QueueHandle_t input_queue = (QueueHandle_t) pvParameters;
int value;
while (1) {
while (xQueueReceive(input_queue, &value, 10) != pdPASS);
printf("%d\n", value);
}
}

void app_main(void) {
/* adc_init() here */
tx_queue = xQueueCreate(TX_BUFFER_SIZE, sizeof(int));
avg_queue = xQueueCreate(AVG_BUFFER_SIZE, sizeof(int));
avg_pair = (stage_interface_t){avg_queue, tx_queue};

xTaskCreate(TaskSample, "Sample", 4096, avg_queue, 0, NULL);
xTaskCreate(TaskAvg, "Avg", 4096, &avg_pair, 0, NULL);
xTaskCreate(TaskTransmit, "Transmit", 4096, tx_queue, 0, NULL);
}

/* Q6: Not always useful to separate avg into a task.
Separate task adds queue overhead & context switching.
Worth it only if avg is slow or you want independent scaling.
Bonus: pre-filling avoids outputting averages computed from
fewer than AVG_WINDOW_SIZE values at startup. */

```

Quest 2: Thresholding — complete pipeline

10XP

```

/* Additions to the running average code above */

#define THRES_BUFFER_SIZE 10
#define THRESHOLD 100

QueueHandle_t thres_queue;
stage_interface_t thres_pair;

void TaskThreshold(void* pvParameters) {
    stage_interface_t* pair = (stage_interface_t*) pvParameters;
    QueueHandle_t input_queue = pair->input;
    QueueHandle_t output_queue = pair->output;

    int last = 1 << 16; /* >> max ADC (4095) + THRESHOLD */

    while (1) {
        int current;
        while (xQueueReceive(input_queue, &current, 10) != pdPASS);

        int diff = current - last;
        if (diff < 0) diff = -diff; /* abs value */

        if (diff > THRESHOLD) {
            while (xQueueSendToBack(output_queue, &current, 10) != pdTRUE);
            last = current;
        }
    }

    /* Updated app_main: */
    void app_main(void) {
        /* adc_init() */
        tx_queue = xQueueCreate(TX_BUFFER_SIZE, sizeof(int));
        avg_queue = xQueueCreate(AVG_BUFFER_SIZE, sizeof(int));
        thres_queue = xQueueCreate(THRES_BUFFER_SIZE, sizeof(int));

        avg_pair = (stage_interface_t){avg_queue, thres_queue};
        thres_pair = (stage_interface_t){thres_queue, tx_queue};

        /* Priority 0 (not 1!) to avoid starving FreeRTOS idle/watchdog */
        xTaskCreate(TaskSample, "Sample", 4096, avg_queue, 0, NULL);
        xTaskCreate(TaskAvg, "Avg", 4096, &avg_pair, 0, NULL);
        xTaskCreate(TaskThreshold, "Threshold", 4096, &thres_pair, 0, NULL);
        xTaskCreate(TaskTransmit, "Transmit", 4096, tx_queue, 0, NULL);
    }

    /* Watchdog fix: priority 1 tasks can starve the FreeRTOS idle task
    which feeds the hardware watchdog timer. Lowering to priority 0
    allows the idle task to run, preventing the WDT reset. */

```